T-Minus_6

D NOVAS · NASA Space Apps Challenge 2026 · Toronto · Nov 14 and 15

Project document, updated Nov 10, 2026 · Team: Dhyanesh, Shubh, Nubaid, Ojas, Vidhan, Aansh

> **How to use this document.** Parts 1 to 3 explain the project. Part 4 is the hardware: every diagram, wire list and step to build it, written for people who have never touched electronics. Part 5 is the plan for the hackathon weekend. Wire numbers in the pictures match the wire lists next to them. Tick the boxes as you go.

Contents

Part 1: The project
Overview · What changed in this version · The hackathon · System overview · Auto-pilot · The game
Part 2: The machines
The robot · Asteroid tower · Weather module · Lives and game over · Robot camera
Part 3: Parts
Parts list · Who brings what
Part 4: Hardware guide
Electronics basics · Circuit diagrams for everything · Weather station · Robot · Tower · Camera · Pin cheat sheet
Part 5: Software and the weekend
Software · Build plan · Hackathon weekend plan · Risks, tests and demo · Open decisions · Sources

Overview

D NOVAS is building **T-Minus_6**, a spacecraft that dodges space junk on its own. It is shown two ways at once: a simulation on a screen, and a small four-wheeled robot on the table. Both are driven by the same Bluetooth gamepad.

The problem. Earth orbit is crowded with debris and rocks, and a spacecraft also swings between extreme heat in direct sun and extreme cold in shadow. It has to notice hazards and react faster than a person can.

The project. A game shows T-Minus_6 orbiting Earth. The robot, also named T-Minus_6, circles a stand-in Earth on a cardboard board and copies every move of the on-screen spacecraft in real time. An auto-pilot mode dodges obstacles by itself. A launcher tower fires slow paper balls at the robot as real “asteroids”. A weather station warns the crew when the spacecraft flies into a hot space or a cold space. A second display shows the 15 lives the spacecraft must survive with; every hit takes one, and at zero the game ends and the robot stops. A camera on the robot lets the player switch to the robot’s own view.

What the demo shows

- Push the gamepad stick left: the on-screen T-Minus_6 and the robot both turn left at the same moment.
- Switch on auto-pilot and both dodge debris and asteroids by themselves, even while the player keeps steering.

- An asteroid visibly flies across the screen while the tower fires a paper ball at the robot.
- Blow hot or cold air at the weather sensor: a warning message tells the crew that the spacecraft is in a hot (or cold) space, then the LED and buzzer blink rapidly.
- Every hit takes one of 15 lives, shown as hearts on the lives display. At zero lives, the game shows a failed-orbit animation and the robot stops, ignoring the gamepad until restart.

Why it stands out. The virtual and physical versions stay in sync, the hazards are real objects you can see and dodge, and the same safety logic runs in both.

Team and event. Team D NOVAS, Toronto local event of the NASA Space Apps Challenge, November 14 and 15, 2026.

What changed in this version

Topic	Now
Name	The spacecraft in the game and the robot are both called **T-Minus_6**.
Controller	A **Bluetooth gamepad** (Xbox style) paired with the laptop. The kit's joystick module and the mission-control controller board are gone.
Boards	**Two Arduino UNO Q** (bought separately): one is the robot brain, one runs the asteroid tower. The **ELEGOO UNO R3** from the kit runs the weather station and the lives display.
Displays	Two. The kit's **16-pin LCD** shows the weather. An **I2C display** shows the lives (15 hearts).
Weather	Hot air or cold air blown at the sensor triggers a crew warning, then the LED and buzzer blink rapidly. Hot and cold use different LEDs and different buzzers.
Robot	**Four wheels** (two per side) with a TB6612FNG motor driver.
Camera	Either an **ESP32-CAM** or a **normal phone**. Both have a diagram in Part 4.
Removed	The Bluetooth link to the robot (it now uses Wi-Fi), the joystick module, the mission-control board, the second MB102 and 9 V battery, the second launcher servo as a default part, and the old wiring and part lists.

Why the old plan had two servos, two power modules and two 9 V batteries, and what we do now

The old tower used **two servos**: one pushed the ball out and one turned the tower to aim. It also used **two MB102 power modules with two 9 V batteries**: one to feed the tower's servos and one to feed the robot's stepper motor. The reason was safety for the boards: servos and motors pull sudden bursts of current, and if they share a supply with the board's brain, the board can reset.

> **New answer:** we need only **one servo** (the push servo, on the tower) and **one MB102 with one 9 V battery** (on the robot, for the turret stepper and the distance sensor). The tower now has its own UNO Q on its own power bank, with a 100 µF capacitor to smooth the servo's bursts, so it needs no MB102 and no 9 V battery. The second (pan) servo is optional; if you add it, it connects to pin D10 and nothing else changes. If you skip the MB102, you can power the stepper from the power bank's 5 V instead, but then watch for resets.

The hackathon

The NASA Space Apps Challenge is a free global hackathon. In 2026 it runs Saturday, November 14 and Sunday, November 15, and D NOVAS is entered at the Toronto Local Event.

Date	What happens
Aug 26	Registration opens
Sep 17	Challenge summaries and team formation open
Oct 28	Challenge statements available
Nov 2	Space Apps Connect opens
Nov 13	Global Offers released; the Project Submission and Judging and Awards guides become available
Nov 14 to 15	The hackathon
Sat Nov 14, 9:00 AM to Sun Nov 15, 11:59 PM (local time)	Project submission window, from the Project tab on the team page
After the hackathon	Judging by experts from NASA, space agency partners and industry
Nov 30	Participant survey due
Dec 2026	Global nominees, finalists and honorable mentions announced
Jan 2027	Global winners announced

Submitting a project earns a participant certificate and makes the team eligible for Global Judging.

Our entry

- Team name: D NOVAS.
- Local event: Toronto. Ask the Local Lead about venue, table space, power outlets and whether hardware demos are welcome.
- Teams have no more than six participants. Teammates join through the team page; untick “Accepting Team Members” once the team is full.
- Challenge: the team page currently shows “Build a Junior Astronaut Mission Trainer”. See Open decisions for whether to keep it or switch to “Space Mission Design Game”. The team can change it until the hackathon closes on Nov 15.

Rules and limits to plan around

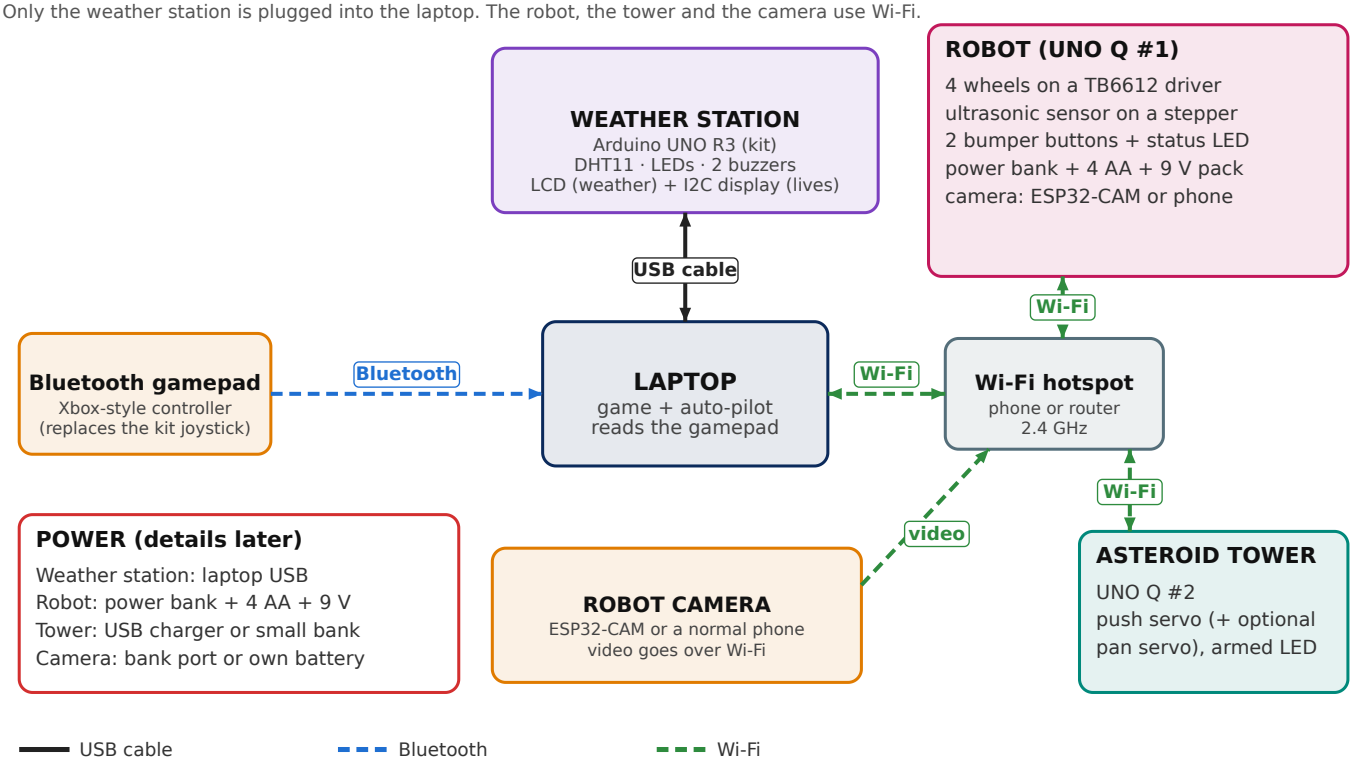
- The event is open to all ages. Participants under 18 must be registered by a parent or guardian and be accompanied by a parent or guardian at all times during the event.
- Recognizable names, voices and likenesses of anyone under 18 should not appear in videos or project submissions. Plan the demo video around the hardware and screen, hands only, with captions or an adult voice-over.
- The team photo cannot include minors or copyrighted or trademarked images. D NOVAS uses a graphic (the spacecraft avatar), not faces.
- The public FAQ does not say whether code or hardware started before the event is allowed, or what a hardware submission must contain. Read the Project Submission guide on Nov 13 and ask the Local Lead before building anything you plan to present as made during the event.

System overview

Four parts work together: the **laptop** (game, the only auto-pilot, and the link to everything), the **weather station** (UNO R3, USB cable), the **robot T-Minus_6** (UNO Q #1, Wi-Fi) and the **asteroid tower** (UNO Q #2, Wi-Fi). The player holds a Bluetooth gamepad that pairs with the laptop.

The whole hardware at a glance

Only the weather station is plugged into the laptop. The robot, the tower and the camera use Wi-Fi.



System map. Solid line = cable, dashed line = wireless. Everything talks through the laptop.

The gamepad goes to the laptop over Bluetooth. The laptop's movement model sends the same speed and turn to the game screen and, over Wi-Fi, to the robot. The robot's distance scan comes back to the one auto-pilot, which also tells the tower when to fire. The weather station talks to the game in both directions over USB.

> **Check on day one.** The UNO Q has a small Linux computer plus a separate microcontroller. Wi-Fi sits on the Linux side, so a short Python program on the UNO Q must pass the laptop's messages to the microcontroller (the Arduino "Bridge"). This is the one part nobody has tested. Fallback: plug a USB-C data cable from the laptop to the UNO Q for the demo.

How one stick drives both

1. The gamepad's left stick is read by the laptop (pygame) about 100 times a second.
2. The laptop turns it into one speed and one turn command.
3. In the same moment, it moves the spacecraft on screen and sends wheel commands to the robot over Wi-Fi.
4. Both follow the same command stream, so left on the stick is left on the screen and on the table.

Speed targets (to be measured, not yet achieved). Under 100 ms from stick to robot wheels over Wi-Fi (the old 50 ms target was for a direct cable, so measure and tune). The robot stops 300 ms after commands stop. Measure with a timestamped test message sent to the robot and echoed back.

Auto-pilot

Auto-pilot makes the spacecraft and the robot dodge hazards and keep orbiting on their own, even while the player is steering with the gamepad.

- It works the same in the game and on the real robot.
- It stays available while a person controls both with the gamepad.
- The spacecraft and the robot follow the same instructions and move on the same path.

Proposed behavior (to confirm). The distances are starting values at table scale and will be tuned during testing.

Situation	What auto-pilot does
No hazard near, no stick input	Holds the orbit: flies around the Earth stand at a fixed distance
No hazard near, player steering	The stick input is applied as a nudge
Hazard inside the warning zone (about 40 cm)	Slows down and starts turning toward the clear side
Hazard inside the danger zone (about 25 cm)	The dodge overrides the stick until the path is clear, then returns to the orbit lane
Anything closer than 8 cm to the robot	The robot stops at once. This hardware safety is always on, even with auto-pilot off

One brain, two bodies. The laptop runs a single auto-pilot. The robot streams its ultrasonic scan to the laptop. The laptop merges those real detections with the game's own hazards into one hazard list, then sends the resulting command (speed and turn) to both the game spacecraft and the robot. Two separate auto-pilots would slowly drift apart, so only one decides.

How a dodge is chosen

1. Build the hazard list: position, speed and size of every hazard.
2. Predict where each one will be over the next 3 seconds. Debris stays put; an asteroid follows its launch path.
3. If the craft's predicted path comes within the danger distance of a hazard, pick the side with the bigger gap, steer that way and slow down.
4. After about 1 second of clear path, return smoothly to the orbit lane.

Why the laptop handles the paper-ball asteroids. A small, quick paper ball is too small and too fast for a sweeping ultrasonic sensor to catch. The laptop knows when the tower fired and where it aimed, so it predicts the ball's path and tells the craft and the robot to dodge. The ultrasonic sensor handles debris sitting on the table.

Controls. One gamepad button (Y) toggles auto-pilot (off at power-up). Its state shows on the game screen and on a status LED on the robot.

The game

The game is a 2D top-down simulation of the spacecraft T-Minus_6 orbiting Earth, built so the robot can copy it move for move.

Screen layout. Earth in the middle, the orbit ring around it, T-Minus_6 on the ring. A heads-up display shows auto-pilot status, speed, distance from the orbit lane, hazard warnings, the temperature and its status from

the weather station, and the lives left as hearts.

Controls (the same ones the robot gets)

Gamepad	What it does
Left stick up / down	Speed forward and back
Left stick left / right	Turn left and right
Y button	Toggles auto-pilot
B button (or key C)	Switches to the robot camera view
Start button	Restart after game over (and re-sync the robot to its start mark)

Button numbers differ between Windows, Mac and Linux. The first test script prints the number of every button you press, and you copy those numbers into the settings file.

One movement model for both. The gamepad gives a speed and a turn rate. The laptop mixes them into left and right wheel speeds (left = speed + turn, right = speed − turn). The game moves the spacecraft with the same speed and turn values. To make the screen and the table match, measure the robot once (centimeters per second at full stick, degrees per second at full turn) and use those numbers in the game.

Hazards

- **Debris:** static pieces on the orbit ring. The team places real pieces on the table to match a scenario file, so the robot's ultrasonic sensor finds them for real and the game draws the same ones.
- **Asteroids:** the game spawns an asteroid in a marked asteroid sector of the ring, shows a one-second warning (flash and sound), and then draws the asteroid flying toward the spacecraft while telling the tower to fire. Target flight time from launch to the orbit lane is 2 to 3 seconds. The asteroid is slow on purpose so a person can dodge it by hand, and the on-screen asteroid uses the paper ball's measured speed.
- **Weather:** the HUD shows the temperature from the weather station. Hot shows a crew warning banner: *"WARNING, CREW: T-Minus_6 is in a HOT space."* Cold shows *"WARNING, CREW: T-Minus_6 is in a COLD space."* The LEDs and buzzers on the table blink at the same moment.

Hits. A hit is a collision with debris or an asteroid, and every hit takes one life (see Lives and game over). The game decides hits from the craft's estimated position, and the robot's front bumper adds a real contact signal for debris. The robot cannot feel a paper ball.

Scoring (proposed). Seconds on orbit without a hit, hazards dodged, and a side-by-side result of a manual run versus an auto-pilot run, which makes a clear demo.

Screens. Start (connection check for the gamepad, robot, tower and weather station, plus calibration), play, game over (the failed-orbit animation), and results.

Optional NASA data. NASA's DONKI space-weather service (api.nasa.gov, free key) lists real solar flares. A real recent flare can trigger a hot-space event in the game, which ties the project to real NASA data.

Tech. Python with pygame for the game and pyserial for the weather station. The gamepad and the serial port are each read in their own thread, separate from the 60 frames-per-second drawing.

The robot: T-Minus_6

The robot is a small **four-wheel** vehicle that circles a stand-in Earth, copies the spacecraft's moves, and scans for hazards with an ultrasonic sensor mounted on a stepper motor.

What the team decided

- A stepper motor turns the ultrasonic sensor instead of a servo, because it moves more smoothly.
- A ball or other round object stands in for Earth, and the robot orbits it.
- The robot and the game spacecraft follow the same instructions, with very fast response.

Body

- **Chassis:** a flat plate about 150 mm wide and 240 mm long with **four wheels**, two on each side. Wheels on the same side always turn together, so the robot steers like a tank: it turns on the spot by spinning the left side and right side in opposite directions.
- **Motors:** four TT gear motors (6 V) with 65 mm wheels. The two left motors are wired in parallel to one output of the TB6612FNG driver; the two right motors to the other output. A 0.1 µF capacitor across each motor's two terminals cuts electrical noise.
- **Motor driver:** a TB6612FNG module. The old L293D chip cannot safely drive four motors, so it is replaced. The TB6612 has two channels (left, right), each good for about 1.2 A, and runs from 3.3 V logic, which suits the UNO Q.
- **Brain:** Arduino UNO Q #1. It has a small Linux computer plus a separate microcontroller for the motors and sensors. It runs on 3.3 V logic, so the distance sensor's 5 V echo signal goes through a two-resistor divider.
- **Wireless:** the UNO Q has Wi-Fi built in, so the laptop sends wheel commands over the hotspot. The Linux side passes them to the microcontroller (see the day-one check in System overview). No Bluetooth module is used anywhere on the robot.
- **Camera:** an ESP32-CAM or a normal phone on a mast (see Robot camera).
- **Power:** four separate supplies share one ground: a power bank for the UNO Q and camera, a 4 AA pack for the motors, and a 9 V battery through an MB102 module for the stepper and the distance sensor. Part 4 draws every connection.

Sensor turret

- A 28BYJ-48 stepper (with its ULN2003 driver board) carries the HC-SR04 ultrasonic sensor.
- It sweeps about 90 degrees ahead (45 degrees each side) and takes a distance reading every few degrees. It can also turn sideways to read the distance to the Earth stand so the robot can hold its orbit radius.
- The 28BYJ-48 is slow, roughly 10 to 15 revolutions per minute, so a 90-degree sweep takes about 1.5 seconds or more. Keep the robot slow (starting target about 10 cm per second) and step the motor without blocking, using the AccelStepper library, so driving and sensing never wait on each other.
- The sensor points straight ahead at power-up, which sets the zero angle.
- The HC-SR04 reads from about 2 cm to 4 m with a beam about 15 degrees wide, and needs about 60 ms between pings. Make debris flat-faced and at least 8 cm tall. Make Earth a round tub or can with a ball on top, because sound bounces off a bare sphere and it can be missed.

What the robot does

1. Receives left and right wheel commands from the laptop about 50 times a second.
2. Sends back its scan (angle, distance) and state about 15 times a second.
3. Stops if no command arrives for 300 ms.
4. Stops if anything is within 8 cm in front, whatever the laptop says. A front bumper with two buttons reports real contact with debris to the laptop, which counts it as a hit.
5. Lights a status LED when auto-pilot is on. On a game-over command it stops, parks the stepper, ignores wheel commands, blinks the LED fast, and stays stopped until the laptop sends a reset.

The arena. The robot drives on a big rectangular cardboard with a raised border, the team's own board. The Earth stand sits in the middle, the debris pieces sit on the orbit ring around it, and the tower stands just outside one edge, aimed across the board. The border keeps the robot and the paper balls on the table and gives the ultrasonic sensor a wall to see, so the auto-pilot treats the border as an obstacle and steers away from it. Starting sizes (measure the real table first): about 120 x 80 cm board, a border about 8 to 10 cm tall made of cardboard strips, an Earth stand about 15 cm across, and an orbit ring about 25 to 30 cm from its centre. Mark the ring and the robot's start spot on the board with tape. The board's outline is also the game's screen boundary, so the screen and the table match.

Track. Earth stand about 12 to 15 cm across. Orbit ring about 40 cm from Earth's center, marked with tape. That needs about 110 cm by 110 cm of table; with less space, use a 30 cm ring. Confirm the table size at the venue.

Drift. The robot tracks its position by dead reckoning (adding up its commanded moves), so after a minute or two it can be a few centimeters off the game. A sync button puts the robot on its start mark and resets the game. An overhead webcam tracking a marker on the robot is an optional upgrade for a true position.

Asteroid tower

The tower is a small structure beside the orbit ring that shoots slow paper balls at the robot, so the asteroids are real objects the player can see and dodge. It has its own brain, the second Arduino UNO Q.

What the team decided

- A servo shoots paper balls or another light material, aimed at the robot.
- The balls are slow, so a person controlling the robot by hand can dodge them.
- In the game, the asteroid visibly travels toward the spacecraft, the same way.

Design

- **Tower:** cardboard, foam board or any light material, about 30 to 40 cm tall, taped or weighted to the table. At the top, a short ramp or tube points at the orbit ring.
- **Feeding:** a hopper above the ramp holds 5 to 10 paper balls. Gravity feeds one ball at a time.
- **Push servo (one SG90, from the kit):** swings a small gate or pusher that releases one ball onto the ramp. A ball rolling down a shallow ramp is slower and more repeatable than a flick.
- **Pan servo (optional, a second SG90):** turns the ramp left and right to aim. Without it, the ramp is aimed by hand and the tower fires along one fixed lane.
- **Controls on the tower:** an armed LED (lit when ready) and a test-fire button that fires without the laptop.
- **Balls:** about 2 to 3 cm, made of tightly crumpled paper or tissue, a few grams each. Use nothing hard, and keep people out of the line of fire.

- **Speed:** start with a shallow ramp and steepen it only until the ball takes 2 to 3 seconds from launch to the orbit lane. Slower is better for the first version.
- **Position:** outside the ring at the edge of the asteroid sector, so the ball crosses the ring at roughly a right angle. Asteroids only launch while the spacecraft is in that sector.

Aiming. The game knows where it estimates the craft will be about 2 seconds from now and (if there is a pan servo) sends the aim angle to it. The robot's real position can be a few centimeters off, so a shot that crosses the lane near the robot counts. Calibrate by firing 10 test shots at 5 aim angles and recording where each lands, then use that table for aiming.

Game link. The game sends an aim command and then a fire command to the tower over Wi-Fi. The tower replies when the ball is released, and the on-screen asteroid starts moving at that moment, so screen and table stay in step.

Power. The tower's UNO Q runs from its own power bank (or a USB-C wall adapter). The servo takes its power from the board's 5 V pin with a 100 µF capacitor across it to smooth the bursts. Do not share this power bank with the robot.

Weather module

The weather station watches the temperature. When the air around the sensor gets too hot or too cold, it warns the crew with a message on the weather display, a blinking LED and a buzzer. It shows a steady green light when everything is normal.

What the team decided

- A DHT11 temperature and humidity sensor feels the air. For the demo and in the game, **hot air** (a hair dryer on low, from a distance) and **cold air** (an ice pack in a sealed bag held near it) stand in for the heat and cold the spacecraft meets in space.
- When the game says the spacecraft is in a hot space (or the sensor reads hot), the crew sees a warning message: on the game screen and on the weather display. Then the **red LED and the active buzzer blink rapidly**.
- Cold works the same way: a warning message, then the **blue LED and the passive buzzer blink rapidly**.
- The two buzzers make different sounds, so the crew can tell hot from cold by ear.
- Normal temperature: the green LED stays on and nothing sounds.
- Every 5 seconds the temperature is recorded in Celsius on the weather display and in the game log.

States

The temperatures are starting values; set the final ones in Open decisions.

State	Condition	LED	Sound	Weather display (16 characters per line)
Normal	18 to 30 C	Green, solid	Silent	Line 1: **T 24.0C NORMAL** Line 2: **H 40%**
Hot	Above 30 C (or the game sends a hot event)	Red, blinks about 5 times a second	Active buzzer: fast beeps	Line 1: **!! HOT SPACE !!** Line 2: **CREW: HEAT WARN**
Cold	Below 18 C (or the game sends a cold event)	Blue, blinks about 5 times a second	Passive buzzer: low tone, fast beeps	Line 1: **!! COLD SPACE !!** Line 2: **CREW: COLD WARN**

The warning text appears on the display **at the moment the state changes**, not at the next 5-second update. While the state is Normal, the display goes back to showing the temperature every 5 seconds. The sounds differ because the buzzers differ: the kit's active buzzer makes one fixed pitch, while the passive buzzer's pitch is set in code with tone(), so use a low pitch for cold. To avoid flicker at the edges, an alarm clears only when the reading is 1 C back inside the normal range. Humidity, the auto-pilot state and the log go to the laptop and the game screen. Each reading is also sent to the laptop so the game saves a log (time, temperature, humidity, state).

Sensor facts. The DHT11 reads 0 to 50 C with about 2 C of error and humidity from 20 to 90 percent. It can be read once a second at most and takes several seconds to react to a change. Keep heat and water away from the wiring, and never put the sensor or its wires in water.

Space weather without touching the sensor. A simulation mode lets the game send a virtual temperature: hot on the sunlit side of the orbit and cold in Earth's shadow. The board treats it like a real reading, so the warnings can be shown on cue.

Code rule. Use timers based on millis(), never delay(). A blinking LED or a beeping buzzer must not stop the serial lines from being read. Read the sensor every 2 seconds so alarms react quickly, and refresh the display and the log every 5 seconds.

Lives and game over

T-Minus_6 starts with 15 lives, every hit takes one, the I2C display shows the lives as hearts, and at zero the game ends in a failed-orbit animation while the robot stops.

What the team decided

- Hitting an obstacle, or being hit by an asteroid, takes 1 life.
- T-Minus_6 starts with 15 lives, and the lives display shows 15 hearts.
- Two displays: the **16-pin LCD** shows the weather and the **I2C display** shows the lives.
- At zero lives the run ends in the game and on the robot. The game shows a failed-orbit animation and the robot stops moving.
- After game over, the gamepad does nothing: the robot does not move and the game does not respond to the stick or any other input, except Start for restart.

Lives display. The I2C 16x2 display shows **15 hearts on line 1**: a full heart for each life left and an outline heart for each life lost. Line 2 reads **T-Minus_6 L15/15**, with the number going down. A display holds 8 custom characters; the two hearts use two of them. On a hit, the newest full heart turns into an outline, the yellow hit LED flashes for 1 second, and the game plays a hit sound. At game over the display reads **SIGNAL LOST** on line 1 and **GAME OVER** on line 2.

What counts as a hit

Hit	How it is detected
Real debris on the table	The robot's front bumper (two buttons) closes, or the game's position estimate overlaps the debris
Virtual debris in the game	The game's position estimate overlaps it
Asteroid (paper ball)	The game decides, from the craft's estimated position, when the ball crosses the lane. A paper ball is too light to sense

After a hit the craft is protected for 1.5 seconds (it flashes on screen), so one collision costs one life instead of ten. The 1.5 seconds is a starting value. Asteroid hits are an estimate, so test and tune the lane width.

Game over sequence

1. Lives reach 0. The laptop freezes the game and ignores the gamepad from now on (except Start).
2. The laptop sends a game-over message to the robot, repeated 5 times a second until the robot confirms. The robot stops all four wheels at once, parks the stepper, ignores any later wheel commands, and blinks its status LED fast.
3. The laptop sends game over to the weather station: the lives display shows GAME OVER, the weather display shows SIGNAL LOST, the warnings go quiet. It also tells the tower to stop firing.
4. The game plays the failed-orbit animation, then the results screen.
5. Restart: pressing Start tells the laptop to reset to 15 lives, and the laptop unlocks the robot and the tower. Put the robot back on its start mark and press sync.

Failed-orbit animation (proposed)

- The engine flame flickers and dies, red warning lights flash, and the screen says “MAIN ENGINE FAILURE”.
- T-Minus_6 starts to tumble and drifts off the orbit ring, whose dashed path breaks into pieces.
- Earth’s pull takes over: the craft falls toward Earth with a trail of sparks, glows orange as it heats up, breaks into fragments, and burns up at the edge of the atmosphere.
- The screen fades to “ORBIT FAILED” with a quiet line, “Signal lost at T+ minutes: seconds”. A low alarm sounds, then silence.
- The results screen shows time survived, hits from debris and from asteroids, and hazards dodged, with a prompt to press Start to restart.

Robot camera (first-person view)

A camera on the robot lets the player see what the robot sees. In the game, the Robot cam button (gamepad B, or key C) switches the main view to the camera picture, with the heads-up display (lives, weather, auto-pilot) drawn over it, or shows it as a small picture next to the map.

What the team decided

- A camera on the robot streams live video to the game.
- The game has an option to show the robot’s view.
- The picture should arrive with as little delay as possible.
- **Two camera options**, to be chosen when the parts arrive: an ESP32-CAM, or a normal phone. The wiring for both is in Part 4.

Build

- **Option A, ESP32-CAM:** a small board with an OV2640 camera and Wi-Fi, about CAD 12 to 25 with a USB programming base. It streams video straight to the laptop over Wi-Fi, so the video does not use the control link. Setup: install the ESP32 board package in the Arduino IDE, open the CameraWebServer example, enter the Wi-Fi name and password, upload, and read the board’s IP address in the serial monitor. The example serves a live stream you can open in a browser.
- **Option B, a normal phone:** install a free camera-streaming app, join the same hotspot as the laptop, and read the address it shows. Put the phone in a holder on the robot with the camera facing forward. No wires.
- **Laptop side:** a camera.py program reads the stream (either option) in its own thread and keeps only the newest frame, so old frames never queue up. The game draws that frame.

- **Mount:** on a mast about 80 mm above the plate, facing forward above the ultrasonic sensor, so the sensor’s sweep never blocks the view.
- **Power (ESP32-CAM):** 5 V from its own USB port on the power bank. Camera boards pull bursts of a few hundred milliamps, so keep the leads short and put a 100 µF capacitor across its 5 V and GND.

About “no delay”. No camera link has zero delay: capturing, compressing, sending and drawing a frame all take time. A plain ESP32-CAM stream often lands somewhere around a tenth to a third of a second (an estimate; measure it). A phone app is usually similar or a bit slower. To keep it as short as it can be:

- Use a small picture (320 by 240), about 15 to 20 frames a second, and JPEG quality around 12 to 15.
- Show only the newest frame and drop the rest.
- Keep the camera off the control link, so a slow picture never slows the wheels.
- Measure it: hold a phone stopwatch in front of the camera, film the screen and the stopwatch together, and read the gap.

Network

- Venue Wi-Fi often stops devices from seeing each other, so do not rely on it. Use a **phone hotspot** (or a small travel router) that the laptop, both UNO Qs and the camera all join. With a phone camera, use a second phone or a router for the hotspot.
- Control messages are tiny, but video is heavy. Test steering and video at the same time; if the stick feels late, lower the picture size or frame rate.

Rule on minors. Point the camera at the table only. The video and any screen recording must not show anyone under 18, so keep people out of its view when recording the demo.

Parts list

The ELEGOO UNO R3 starter kit (Dhyanesh has it) covers most of the electronics. The two Arduino UNO Q boards were bought separately. Everything else is bought and split among all six teammates (see Who brings what). The kit contents below are read from the kit's own parts sheet. This is the correct, up-to-date list.

From the kit and already owned: Dhyanesh

Part	Qty	Used for
Arduino UNO Q (bought separately, not in the kit)	2	#1 robot brain, #2 asteroid tower brain. Both have built-in Wi-Fi
ELEGOO UNO R3 board (from the kit)	1	Weather station and lives display
LCD1602 module (16-pin)	1	Weather display
I2C 16x2 LCD display (added to the list; to buy if it is not in the kit)	1	Lives display (15 hearts)
10 kilo-ohm potentiometer	1	Weather display contrast
Buttons	5	Two robot bumpers and the tower test-fire button (2 spare)
DHT11 temperature and humidity module	1	Weather sensor
LEDs (red, green, blue, yellow, white)	25	Weather: green, blue, red, yellow (hit). Robot status LED. Tower armed LED
Resistors	120	220 ohm for LEDs and the display backlight, 1 kilo-ohm and 2 kilo-ohm for the transistor base and the echo divider (a 2 kilo-ohm is two 1 kilo-ohm in series)
Active buzzer	1	Hot warning
Passive buzzer	1	Cold warning
NPN transistor PN2222	2	Drives the active buzzer
Servo motor SG90	1	Tower push servo
Stepper motor 28BYJ-48 and ULN2003 driver	1 each	Robot sensor turret
Ultrasonic sensor HC-SR04	1	Obstacle sensing
830 tie-point breadboard	1	Weather station wiring
Breadboard jumper wires and female-to-male Dupont wires	65 and 10	Connections
Power supply module (MB102)	1	Steady 5 V for the robot's stepper and distance sensor
9 V battery with snap-on clip	1	Powers the MB102
USB cable (type A to B)	1	Weather station to the laptop

Not needed for this project: the kit's joystick module (the Bluetooth gamepad replaces it), the L293D chip (the TB6612FNG replaces it), relay, IR receiver and remote, 74HC595, 7-segment displays, tilt switch, thermistor, photoresistors, diodes, RGB LEDs and the fan motor.

To buy

Prices are rough Canadian-dollar guesses from memory. Check current prices.

Part	Qty	Used for	Approx. cost (CAD)	Who brings it
TT gear motors with 65 mm wheels	4	Robot drive (4 wheels)	10 to 18	Shubh
TB6612FNG motor driver module	1	Drives the 4 motors	5 to 10	Shubh
Female-to-male Dupont wires	40	Robot wiring (the kit has only 10)	4 to 6	Shubh
Bluetooth gamepad (Xbox style)	1	Controls the game and the robot	0 (borrow one)	Shubh
4 AA battery holder with switch, and rechargeable AA batteries	1 and 4	Motor power	10 to 20	Nubaid
Capacitors: 100 microfarad and 0.1 microfarad	3 and 6	Motor supply, camera, tower servo; one 0.1 per motor plus 2 spare	3 to 6	Nubaid
Mini breadboard (170 tie points)	1	Robot wiring (shared ground and 3.3 V rails)	2 to 4	Nubaid
ESP32-CAM with USB programming base (or a normal phone with a holder)	1	Robot camera	12 to 25 (phone: 0)	Ojas
Power bank with a USB-C and a USB-A port	1	Robot: UNO Q #1 and camera	0 to 25	Ojas
Power bank (or a USB-C wall adapter)	1	Tower: UNO Q #2	0 to 25	Vidhan
Big cardboard or foam board, paper, tape	1 set	Arena board, debris, paper balls	5 to 15	Vidhan
USB-C data cables	2	UNO Q power and programming (one per UNO Q, if they came with none)	0 to 20	Dhyanesh
I2C 16x2 LCD display	1	Lives display	5 to 8	Dhyanesh
Zip ties, hot glue, double-sided tape, velcro, 2 mm foam, robot body plate	1 set	Assembly and the bumper	10 to 20	Aansh
Round tub or can (about 70 mm across) and a foam ball	1	Earth stand	0 to 10	Aansh

The kit’s buttons are reused for the two bumpers and the tower test button. The weather display needs no extra part: the kit’s 16-pin LCD and potentiometer are enough. **Total: about CAD 66 to 212** (the gamepad and a phone camera are free if borrowed).

Optional upgrades

- A second SG90 servo for the tower’s pan (aiming) axis (about CAD 6 to 10, connects to D10).
- MPU6050 motion sensor for better turn tracking.
- Overhead webcam and a printed marker on the robot for true position.
- A second HC-SR04 ultrasonic sensor pointing forward for backup.

Who brings what (about equal for all six)

This list matches the Parts list above, row for row.

Teammate	Brings	Approx. cost (CAD)	Matching job
Dhyanesh	The ELEGOO kit (UNO R3, LCD, MB102 and 9 V, LEDs, buzzers, DHT11, servo, stepper, sensor), both Arduino UNO Q, the laptop; I2C display; 2 USB-C cables	5 to 28	Game, auto-pilot, weather station
Shubh	4 TT motors with wheels; TB6612FNG driver; 40 female-to-male Dupont wires; the Bluetooth gamepad	19 to 34	Robot drive and firmware
Nubaid	4 AA holder with switch and 4 rechargeable AA; capacitors; mini breadboard	15 to 30	Power, wiring and the bumper
Ojas	ESP32-CAM with USB base (or a phone and holder); power bank for the robot	12 to 50	Camera view in the game
Vidhan	Power bank or USB-C adapter for the tower; cardboard, paper, tape	5 to 40	Asteroid tower
Aansh	Assembly set (zip ties, glue, tape, velcro, foam, robot body plate); tub and foam ball	10 to 30	Assembly, arena board and the Earth stand

Each person buys only their own row. Everything reaches Dhyanesh by Wed Nov 11, so the whole robot can be assembled and tested before the Nov 12 freeze. Costs are rough guesses. Keep the receipts and share any difference at the end.

Part 4: Hardware guide

This part is for everyone, including people who have never built a circuit. Read “Electronics basics” first (one page). Then follow the build guide for your module: wire numbers in each picture match the wire list under it. Do the weather station first, because it is the easiest and teaches everything the others need.

Electronics basics (read this first)

Word	What it means
Pin	A metal connection point on a board or part. Boards label them D2, D3 ... (digital), A0 ... (analog), 5V, 3V3, GND.
5V / 3V3	Power pins. The red flags in the diagrams. 5V and 3V3 are different voltages: never connect a 5V thing to a 3V3 pin. The UNO R3 (weather station) pins are 5 V. The UNO Q pins are 3.3 V, which is why the robot echo wire has two resistors.
GND (ground)	The return path for electricity, like the minus of a battery. The black flags. Every part needs its GND connected, and all GND points on one machine are joined together.
Breadboard	A plastic board with holes. The holes in one short column (5 holes) are joined inside. The long rails along the edge (marked + and −) are joined along their whole length. Use the + rail for 5V and the − rail for GND. Push parts and wires in gently. No soldering.
Jumper wire	A wire with metal ends. Use colors: **red** = power, **black** = GND, any other color = signals.
LED	A light. It only works one way: the **long leg is +**, the short leg is −. Always put a 220 ohm resistor in line, or the LED burns out.
Resistor	Limits current. It has no direction. 220 ohm = red, red, brown stripes. 1 kilo-ohm = brown, black, red. 2 kilo-ohm = red, black, red.
Capacitor	A tiny buffer that smooths power. The big one (100 µF) has a stripe on the − leg: the stripe leg goes to GND. Putting it backward can make it pop.
Flag labels	In the diagrams, two flags with the same name (for example two “GND” flags) are joined by a wire. This keeps the pictures clean.

Five rules that save the project

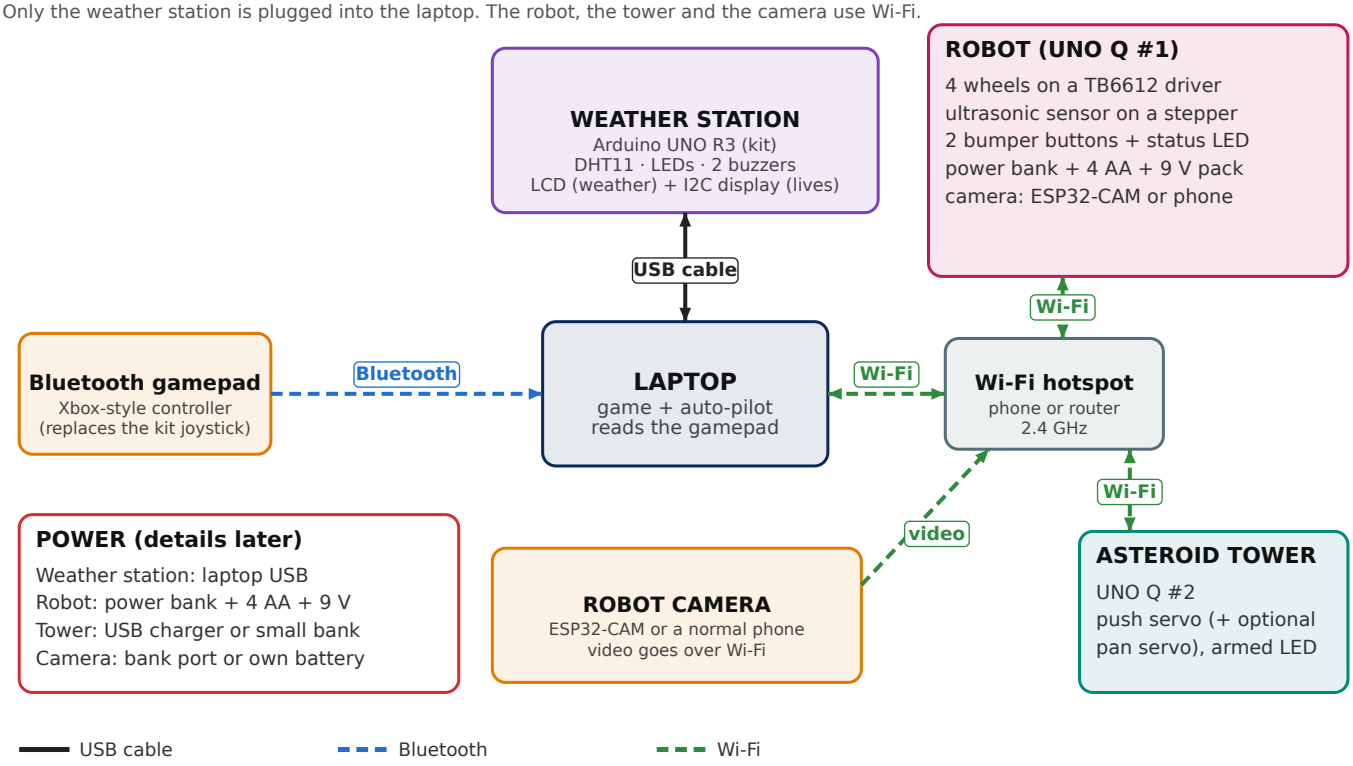
1. Wire with **everything unplugged**. Plug in power last.
2. Check each wire against the list, then tick its box.
3. Never join two **different** power supplies together. Only their GND wires join. Red wires of different supplies never touch.
4. If anything gets hot, smells, or a chip feels warm: unplug power at once.
5. Wrong pin on a part? Do not guess. Check the cheat sheet at the end of this part or ask Dhyanesh.

Circuit diagrams for the whole hardware

These pictures show the complete hardware: how the pieces connect, then how power reaches every piece, then the power on the robot alone.

The whole hardware at a glance

Only the weather station is plugged into the laptop. The robot, the tower and the camera use Wi-Fi.



Whole hardware, big picture: who talks to whom and over which link.

Where the power comes from

Red arrows carry power. Each group has its own source; the only thing they share is ground.

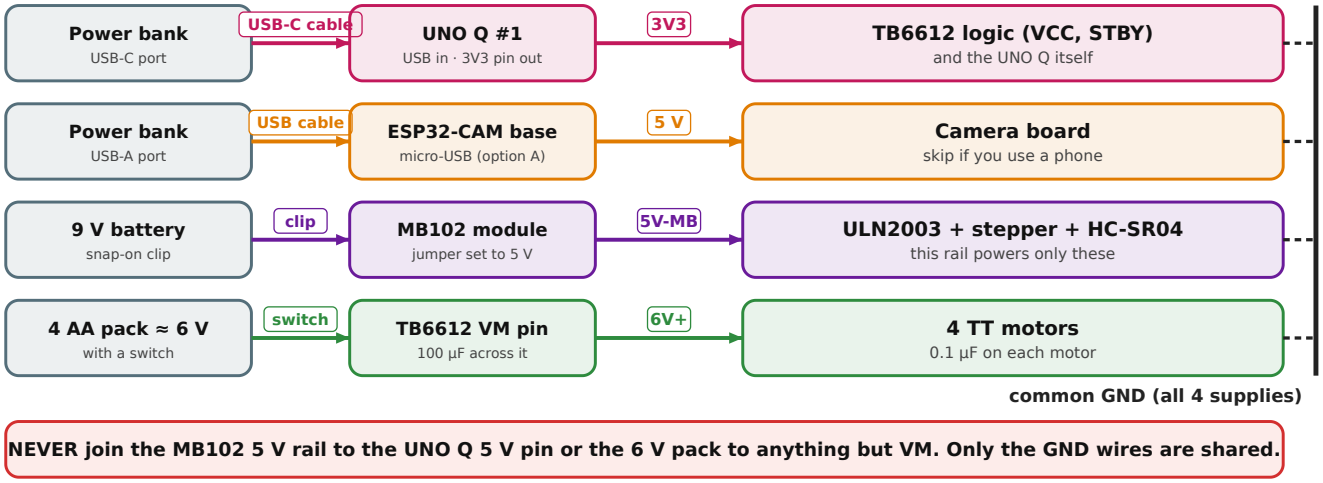
1 Weather station



2 Asteroid tower



3 Robot (four separate supplies, one shared ground)

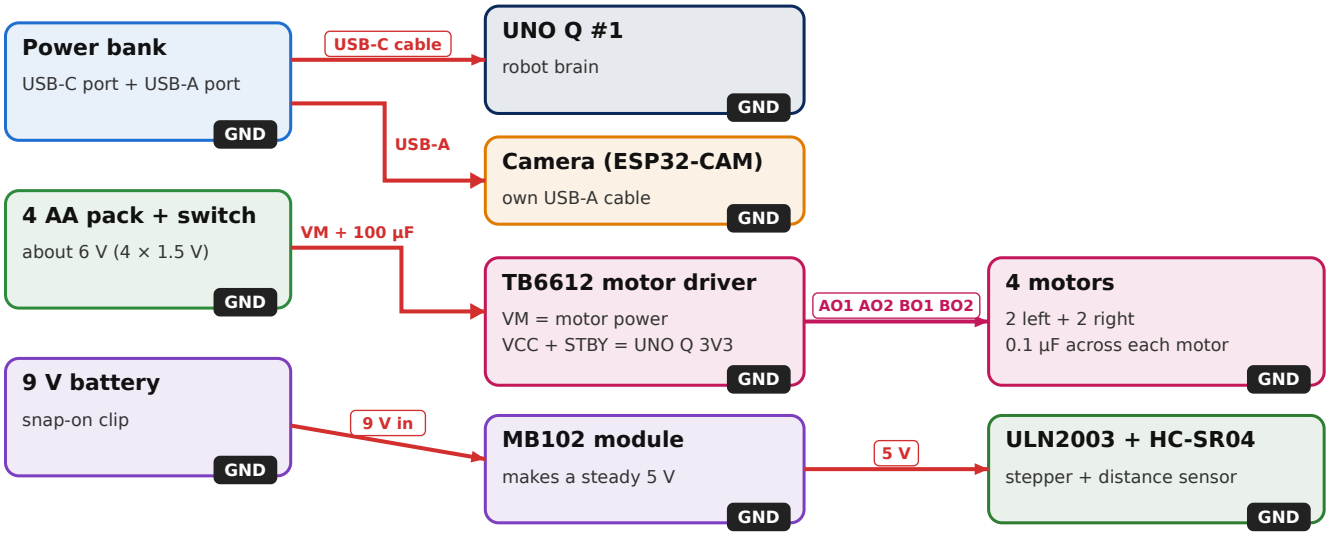


NEVER join the MB102 5 V rail to the UNO Q 5 V pin or the 6 V pack to anything but VM. Only the GND wires are shared.

Whole hardware, power: each machine has its own supply. The machines share nothing but the laptop connection.

Robot power circuit: four supplies, one common ground

Red = power. Black = ground. Each supply feeds only the parts shown.



Every black GND flag is the same wire: join them all with black wire (breadboard GND rail).

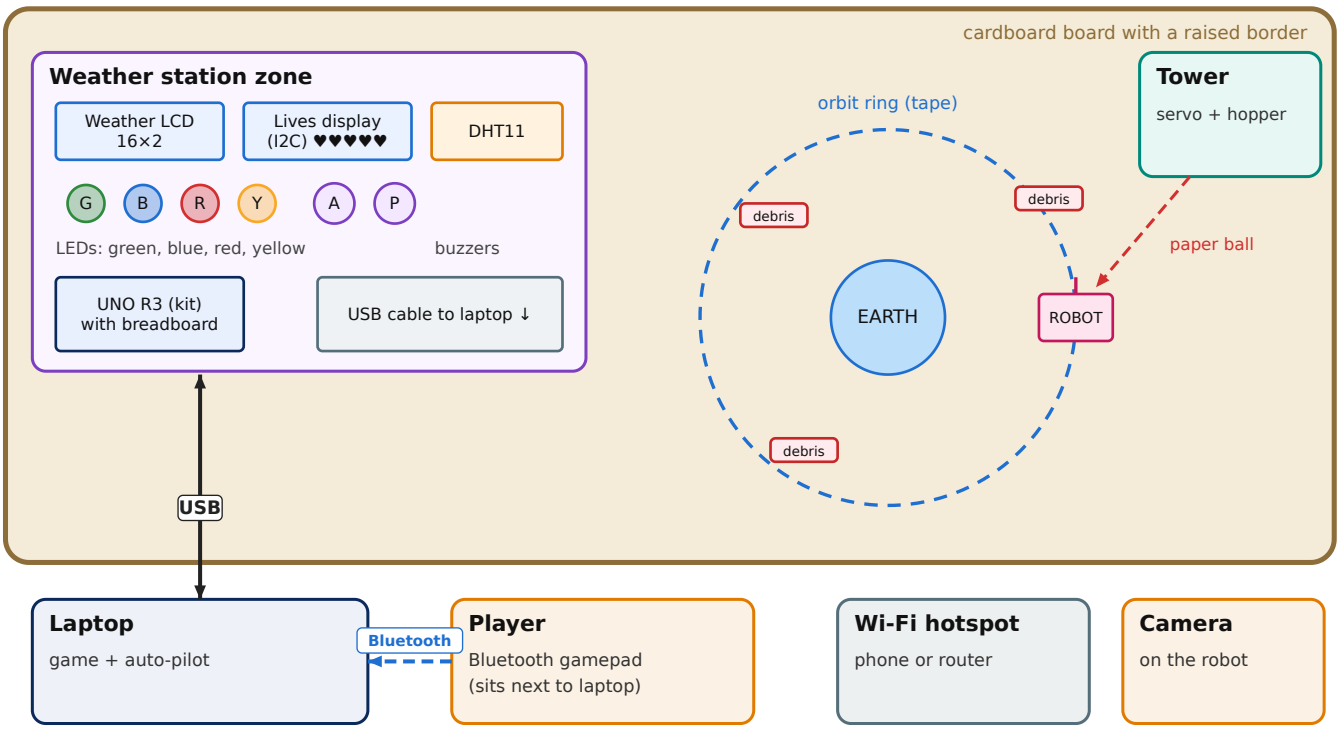
NEVER join the MB102 5 V to the UNO Q 5 V pin, and never put the 4 AA (+) anywhere but VM.
Different supplies share ONLY the black ground. Red wires of different supplies never touch.

Robot power circuit in detail: four supplies, one shared ground.

Layout of everything on the board

Everything on one bordered cardboard board

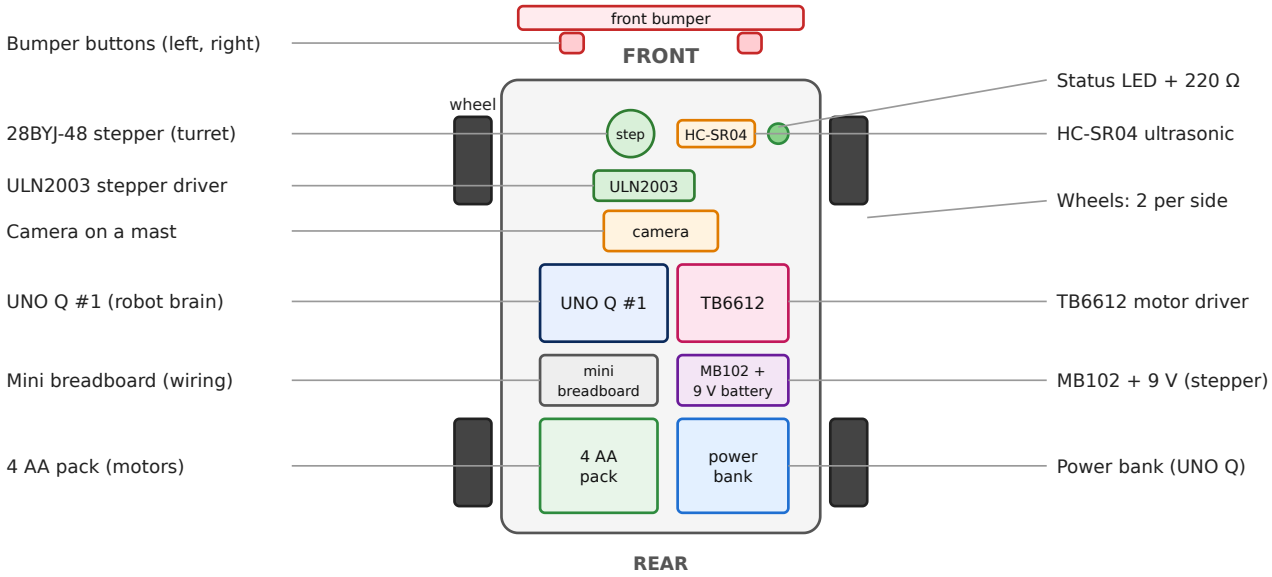
Top view. The board outline is also the game screen boundary.



Where the pieces sit. The weather station goes at the edge of the board near the laptop so the USB cable is short.

The robot from above, every part labeled

Four wheels (two per side). Positions are a suggestion: adjust when the parts arrive.



The robot from above, with four wheels.

Build 1: The weather station (UNO R3)

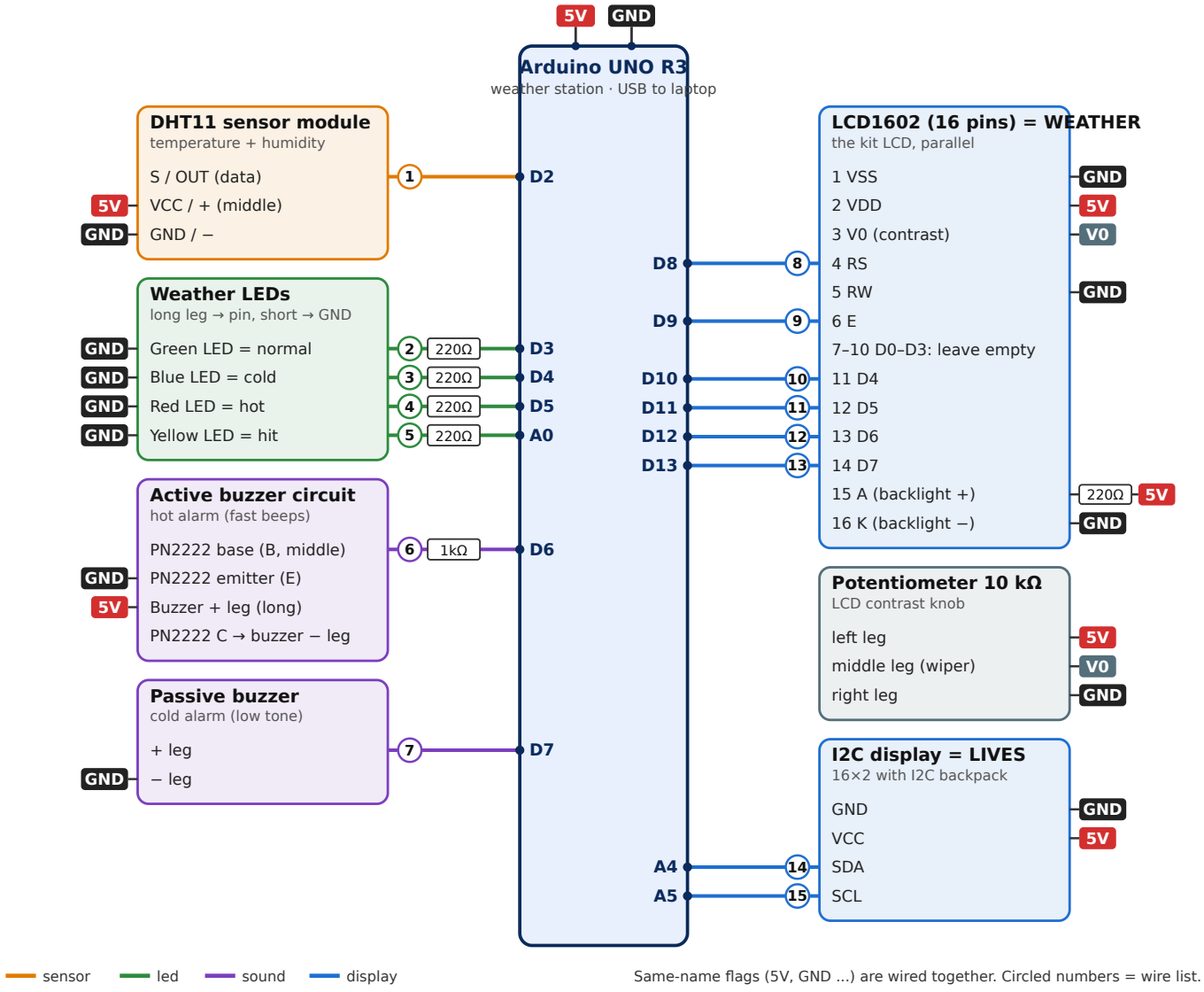
What it is. The ELEGOO UNO R3 with a breadboard. It reads the DHT11 air sensor, lights the green, blue, red and yellow LEDs, runs the two buzzers, shows the weather on the 16-pin LCD and the lives on the I2C display. It plugs into the laptop with the USB cable. It uses 15 signal wires, listed below.

You need

UNO R3, 830 breadboard, USB cable, DHT11 module, 4 LEDs (green, blue, red, yellow), 4 resistors of 220 ohm, 1 resistor of 1 kilo-ohm, 1 resistor of 220 ohm for the backlight, PN2222 transistor, active buzzer, passive buzzer, LCD1602 (16 pins), 10 kilo-ohm potentiometer, I2C display, about 30 jumper wires.

Weather station wiring (UNO R3)

Every wire for the weather station and the lives display. Wire numbers match the list below.



Weather station wiring. Orange = sensor, green = LEDs, purple = buzzers, blue = displays.

Step by step

1. **Power rails.** Unplug the USB cable. Run a red wire from the UNO **5V** pin to the breadboard + rail, and a black wire from a UNO **GND** pin to the − rail.

2. **DHT11 (wire 1).** Plug it on the breadboard. Its three pins are + (5V), OUT (data) and − (GND), but the order differs between makers, so **read the tiny printed labels**. Data wire to **D2**.
3. **LEDs (wires 2 to 5).** Each LED: long leg through a 220 ohm resistor to its pin, short leg to the − rail. Green D3, blue D4, red D5, yellow A0.
4. **Active buzzer circuit (wire 6).** PN2222 with the flat face toward you: legs from the left are E, B, C. E to GND. B through the 1 kilo-ohm resistor to **D6**. Buzzer + leg to 5V, buzzer − leg to the transistor's C leg.
5. **Passive buzzer (wire 7).** + leg to **D7**, other leg to GND.
6. **16-pin LCD (wires 8 to 13).** Follow the table: VSS and RW to GND, VDD to 5V, V0 to the potentiometer's middle leg, RS to D8, E to D9, the LCD pins D4, D5, D6, D7 to UNO pins D10, D11, D12, D13 in that order, A (backlight +) through 220 ohm to 5V, K to GND. Leave D0 to D3 empty. The potentiometer's outer legs go to 5V and GND.
7. **I2C display (wires 14 and 15).** Four wires: GND, VCC to 5V, SDA to **A4**, SCL to **A5**.
8. **Check.** Go down the wire list and tick every box. Then plug in the USB cable.

#	From (part and pin)	To pin on UNO R3	Resistor on the wire	Note	Done
1	DHT11 sensor module: S / OUT (data)	**D2**		data wire	☐
2	Weather LEDs: Green LED = normal	**D3**	220Ω		☐
3	Weather LEDs: Blue LED = cold	**D4**	220Ω		☐
4	Weather LEDs: Red LED = hot	**D5**	220Ω		☐
5	Weather LEDs: Yellow LED = hit	**A0**	220Ω	A0 used as a normal pin	☐
6	Active buzzer circuit: PN2222 base (B, middle)	**D6**	1kΩ	through 1 kΩ	☐
7	Passive buzzer: + leg	**D7**		tone() makes the sound	☐
8	LCD1602 (16 pins) = WEATHER: 4 RS	**D8**			☐
9	LCD1602 (16 pins) = WEATHER: 6 E	**D9**			☐
10	LCD1602 (16 pins) = WEATHER: 11 D4	**D10**			☐
11	LCD1602 (16 pins) = WEATHER: 12 D5	**D11**			☐
12	LCD1602 (16 pins) = WEATHER: 13 D6	**D12**			☐
13	LCD1602 (16 pins) = WEATHER: 14 D7	**D13**			☐
14	I2C display = LIVES: SDA	**A4**			☐
15	I2C display = LIVES: SCL	**A5**			☐

Power and ground connections for the weather station

Part	Pin / leg	Goes to	Resistor
DHT11 sensor module	VCC / + (middle)	**5V**	
DHT11 sensor module	GND / −	**GND**	

Part	Pin / leg	Goes to	Resistor
Weather LEDs	Green LED = normal	GND	
Weather LEDs	Blue LED = cold	GND	
Weather LEDs	Red LED = hot	GND	
Weather LEDs	Yellow LED = hit	GND	
Active buzzer circuit	PN2222 emitter (E)	GND	
Active buzzer circuit	Buzzer + leg (long)	5V	
Passive buzzer	− leg	GND	
LCD1602 (16 pins) = WEATHER	1 VSS	GND	
LCD1602 (16 pins) = WEATHER	2 VDD	5V	
LCD1602 (16 pins) = WEATHER	3 V0 (contrast)	V0	
LCD1602 (16 pins) = WEATHER	5 RW	GND	
LCD1602 (16 pins) = WEATHER	15 A (backlight +)	5V	220Ω
LCD1602 (16 pins) = WEATHER	16 K (backlight −)	GND	
Potentiometer 10 kΩ	left leg	5V	
Potentiometer 10 kΩ	middle leg (wiper)	V0	
Potentiometer 10 kΩ	right leg	GND	
I2C display = LIVES	GND	GND	
I2C display = LIVES	VCC	5V	

Test it

- ☐ The displays light up. Turn the LCD contrast knob (potentiometer) until letters show. On the I2C display turn the small blue screw on the back.
- ☐ Run the I2C scanner example. It should find **0x27**. If it finds 0x3F, change the address in the code.
- ☐ Green LED on at room temperature.
- ☐ Blow warm air from a hair dryer (low, 30 cm away): the LCD shows **!! HOT SPACE !!**, the red LED blinks fast, the active buzzer beeps fast.
- ☐ Hold a cold pack (sealed bag) near the sensor: **!! COLD SPACE !!**, the blue LED blinks fast, the passive buzzer beeps low.
- ☐ Nothing smells hot. The yellow LED flashes when the code says a hit happened.

Build 2: The robot T-Minus_6 (UNO Q #1)

What it is. The robot's brain. It drives four wheels through the TB6612 driver, turns the ultrasonic sensor on a stepper, reads two bumper buttons, shows auto-pilot on a status LED, and talks to the laptop over Wi-Fi. It uses 15 signal wires.

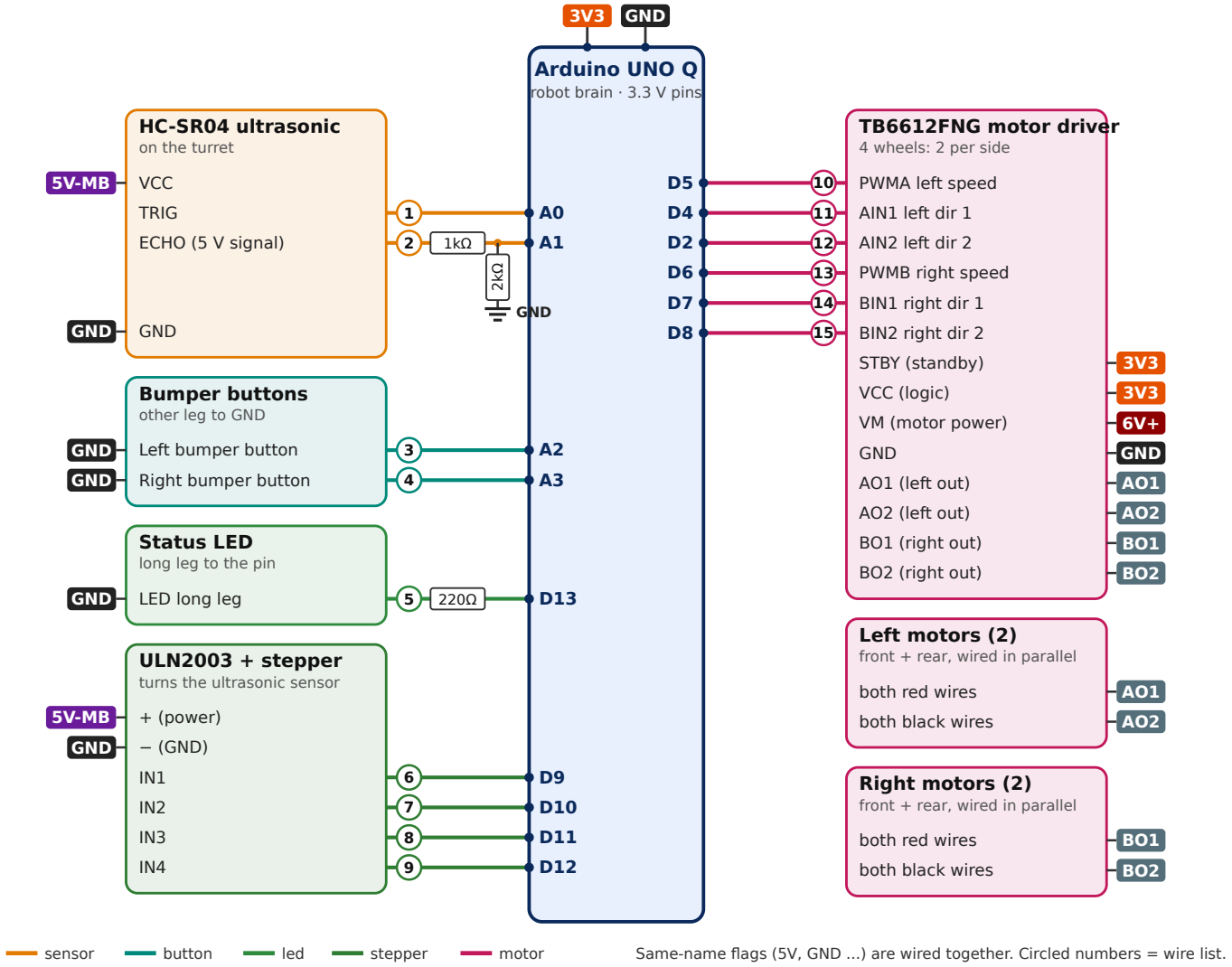
You need

UNO Q #1, TB6612FNG module, 4 TT motors, 4 AA pack with switch (and 4 AA), 9 V battery, MB102 module, power bank with USB-C and USB-A ports, ULN2003 board with 28BYJ-48 stepper, HC-SR04, 2 buttons, 1 LED, 220 ohm, 1 kilo-ohm and 2 kilo-ohm resistors, 100 µF capacitor, six 0.1 µF capacitors, mini breadboard, about 40 female-to-male wires.

> **UNO Q has 3.3 V pins.** Never feed a 5 V signal into a pin. The HC-SR04 ECHO wire sends 5 V, so it goes through the 1 kilo-ohm + 2 kilo-ohm divider shown in the diagram. Power the TB6612 logic (VCC and STBY) from the UNO Q **3V3** pin, not 5V.

Robot signal wiring (UNO Q #1)

15 signal wires from the robot board to its parts. Power wires are shown as flags.



Robot signal wiring: every signal wire from the UNO Q to its part. Power and ground are shown as flags.

Step by step

1. **Mount parts first.** Fix the four motors (two per side), wheels, the plate, the power bank, the AA pack, the stepper turret, the bumper and the camera mast with zip ties, tape or glue. Wires are easier after parts are fixed.
2. **Mini breadboard rails.** One rail is **GND** (black wires), one rail is **3V3** (from the UNO Q 3V3 pin). This is the shared ground that every supply connects to in step 8.
3. **Motor noise capacitors.** Solder or twist a 0.1 µF capacitor across the two terminals of each motor. No polarity.
4. **Motors to the driver.** Both left motors' red wires to AO1, both black wires to AO2. Both right motors' red wires to BO1, black wires to BO2. If the robot later drives backward when you want forward, swap the red and black on that side.
5. **Driver signal wires (wires 10 to 15).** D5 to PWMA, D4 to AIN1, D2 to AIN2, D6 to PWMB, D7 to BIN1, D8 to BIN2. STBY and VCC to 3V3. GND to GND.
6. **Motor power.** The 4 AA pack + (through its switch) to the driver's **VM**, with the 100 µF capacitor across VM and GND (stripe to GND). The pack's − to the GND rail.
7. **Turret and sensor (wires 1, 2, 6 to 9).** Plug the stepper's white plug into the ULN2003. IN1 to D9, IN2 to D10, IN3 to D11, IN4 to D12. HC-SR04: TRIG to A0; ECHO through the divider to A1 (1 kilo-ohm in series, then 2 kilo-ohm from that point to GND). Both get + from the **MB102 5V**.
8. **Stepper power.** Snap the 9 V battery on the MB102 and set its jumper to 5V. MB102 5V to ULN2003 + and HC-SR04 VCC. MB102 GND to the GND rail. **Do not connect MB102 5V to the UNO Q.**
9. **Bumper and LED (wires 3, 4 and 5).** Left bumper button between A2 and GND, right between A3 and GND. Status LED: long leg through 220 ohm to D13, short leg to GND.
10. **UNO Q power.** USB-C from the power bank. The camera uses the bank's USB-A port.
11. **Check.** Tick every wire. Lift the robot so the wheels spin in the air for the first motor test.

#	From (part and pin)	To pin on UNO Q #1	Resistor on the wire	Note	Done
1	HC-SR04 ultrasonic: TRIG	**A0**			☐
2	HC-SR04 ultrasonic: ECHO (5 V signal)	**A1**	1kΩ + 2kΩ from this wire to GND (after the 1kΩ)	voltage divider: protects the 3.3 V pin	☐
3	Bumper buttons: Left bumper button	**A2**			☐
4	Bumper buttons: Right bumper button	**A3**			☐
5	Status LED: LED long leg	**D13**	220Ω		☐
6	ULN2003 + stepper: IN1	**D9**			☐
7	ULN2003 + stepper: IN2	**D10**			☐
8	ULN2003 + stepper: IN3	**D11**			☐
9	ULN2003 + stepper: IN4	**D12**			☐
10	TB6612FNG motor driver: PWMA left speed	**D5**		PWM pin	☐
11	TB6612FNG motor driver: AIN1 left dir 1	**D4**			☐

#	From (part and pin)	To pin on UNO Q #1	Resistor on the wire	Note	Done
12	TB6612FNG motor driver: AIN2 left dir 2	**D2**			☐
13	TB6612FNG motor driver: PWMB right speed	**D6**		PWM pin	☐
14	TB6612FNG motor driver: BIN1 right dir 1	**D7**			☐
15	TB6612FNG motor driver: BIN2 right dir 2	**D8**			☐

Power and ground connections for the robot

Part	Pin / leg	Goes to	Resistor
HC-SR04 ultrasonic	VCC	**5V-MB**	
HC-SR04 ultrasonic	GND	**GND**	
Bumper buttons	Left bumper button	**GND**	
Bumper buttons	Right bumper button	**GND**	
Status LED	LED long leg	**GND**	
ULN2003 + stepper	+ (power)	**5V-MB**	
ULN2003 + stepper	− (GND)	**GND**	
TB6612FNG motor driver	STBY (standby)	**3V3**	
TB6612FNG motor driver	VCC (logic)	**3V3**	
TB6612FNG motor driver	VM (motor power)	**6V+**	
TB6612FNG motor driver	GND	**GND**	
TB6612FNG motor driver	AO1 (left out)	**AO1**	
TB6612FNG motor driver	AO2 (left out)	**AO2**	
TB6612FNG motor driver	BO1 (right out)	**BO1**	
TB6612FNG motor driver	BO2 (right out)	**BO2**	
Left motors (2)	both red wires	**AO1**	
Left motors (2)	both black wires	**AO2**	
Right motors (2)	both red wires	**BO1**	
Right motors (2)	both black wires	**BO2**	

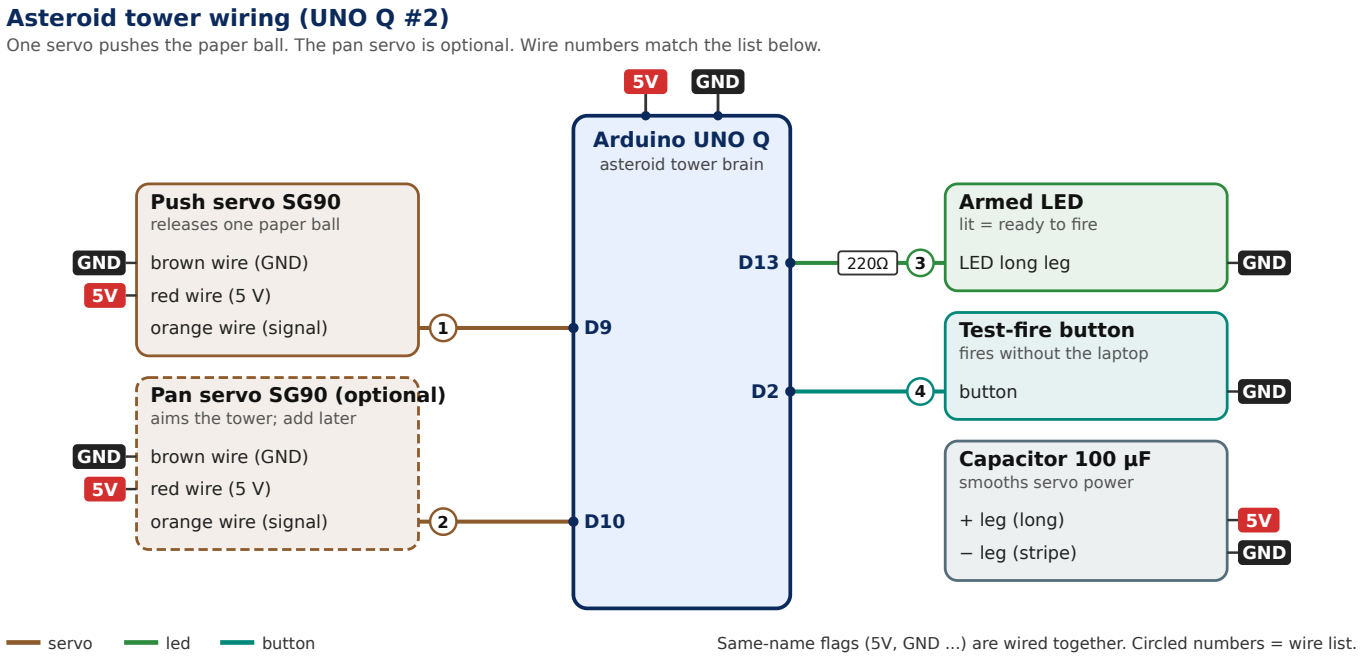
Test it (wheels in the air)

- ☐ UNO Q boots and joins the hotspot.
- ☐ Left side forward, then back. Right side forward, then back. All four wheels turn the same way for “forward”.
- ☐ Stepper sweeps 45 degrees each side and returns.
- ☐ Ultrasonic sensor reads about 10 cm when your hand is 10 cm away.

- ☐ Pressing each bumper button is seen by the code. The status LED lights when auto-pilot is on.
- ☐ Unplug the Wi-Fi: the wheels stop within 300 ms.

Build 3: The asteroid tower (UNO Q #2)

What it is. A small tower with one servo that pushes a paper ball down a ramp. The second UNO Q controls it over Wi-Fi. It uses 3 signal wires (4 with the optional pan servo), so it is the quickest build.



Tower wiring. The optional pan servo is drawn with a dashed box.

Step by step

1. **Servo (wire 1).** SG90 wires: brown = GND, red = 5V, orange = signal. Orange to **D9**.
2. **Capacitor.** 100 µF across 5V and GND, long leg to 5V, striped leg to GND, as close to the servo as you can.
3. **Armed LED (wire 3).** Long leg through 220 ohm to **D13**, short leg to GND.
4. **Test button (wire 4).** One leg to **D2**, other leg to GND.
5. **Power.** USB-C from the tower's own power bank. Do not share it with the robot.
6. **Optional pan servo (wire 2).** Same three wires, orange to D10.

#	From (part and pin)	To pin on UNO Q #2	Resistor on the wire	Note	Done
1	Push servo SG90: orange wire (signal)	**D9**			☐
2	Pan servo SG90 (optional): orange wire (signal)	**D10**			☐
3	Armed LED: LED long leg	**D13**	220Ω		☐
4	Test-fire button: button	**D2**			☐

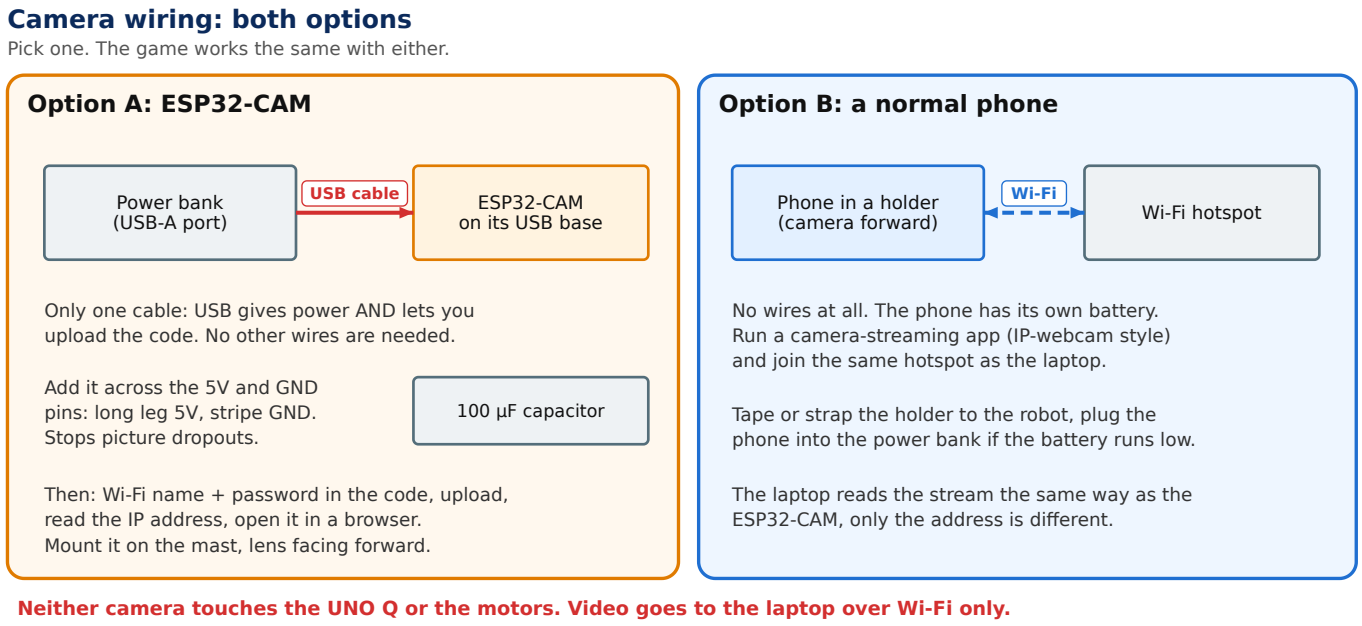
Power and ground connections for the tower

Part	Pin / leg	Goes to	Resistor
Push servo SG90	brown wire (GND)	GND	
Push servo SG90	red wire (5 V)	5V	
Pan servo SG90 (optional)	brown wire (GND)	GND	
Pan servo SG90 (optional)	red wire (5 V)	5V	
Armed LED	LED long leg	GND	
Test-fire button	button	GND	
Capacitor 100 µF	+ leg (long)	5V	
Capacitor 100 µF	− leg (stripe)	GND	

Test it

- ☐ Armed LED is on after boot.
- ☐ Pressing the test button swings the servo once and drops exactly one paper ball.
- ☐ The servo does not reset the board.
- ☐ Tape the ramp so the ball takes 2 to 3 seconds to reach the lane.

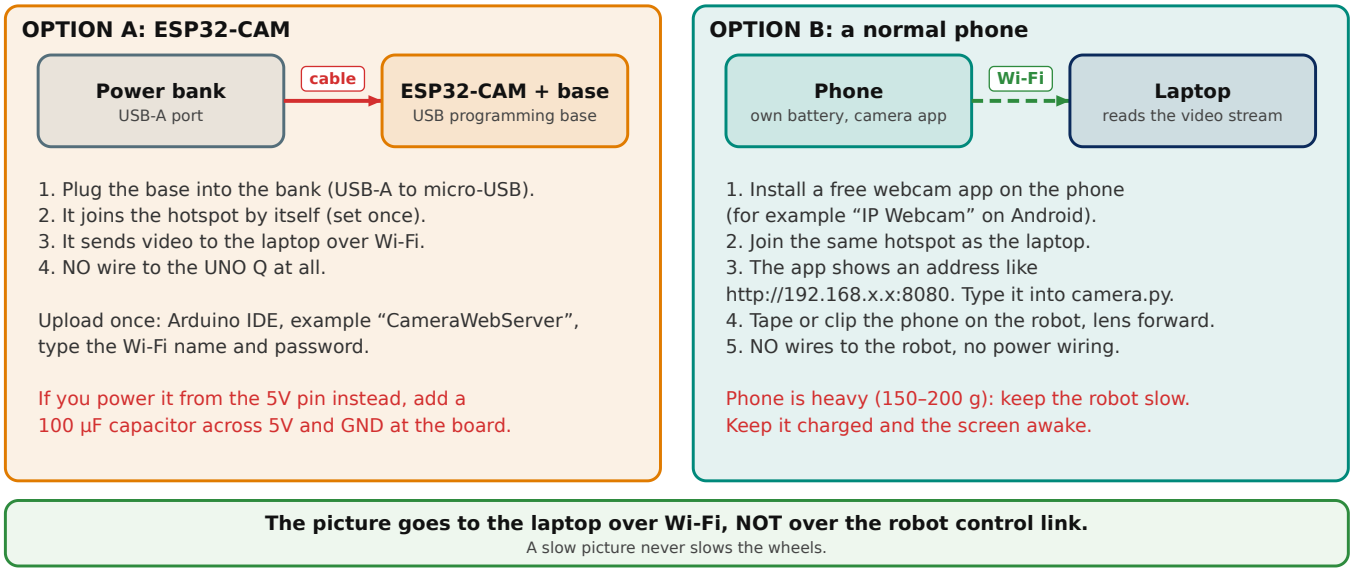
Build 4: The camera (both options)



Camera wiring for both options. Neither connects to the UNO Q.

Robot camera: two options

Pick one. Only option A needs a power cable; neither needs signal wires.



How the camera fits in the whole system.

- ☐ Option A: ESP32-CAM on its USB base, cable to the power bank’s USB-A port, 100 µF across 5V and GND. Upload CameraWebServer, read the IP address.
- ☐ Option B: phone in a holder, camera forward, streaming app on, same hotspot as the laptop.
- ☐ The camera picture appears in the game. Camera on the mast sits above the sensor so the sweep never blocks it.

Pin cheat sheet

Use this to find which pin on a part is which, then match it to the wire lists.

Pin cheat sheet

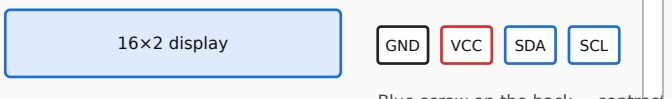
What the pins on each part are called, so you can match them to the diagrams.

LCD1602 (the 16-pin kit LCD): 16 pins along the top edge, pin 1 on the left when the text faces you



1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
VSS	VDD	V0	RS	RW	E	D0	D1	D2	D3	D4	D5	D6	D7	A	K

I2C display: 4 pins



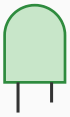
Blue screw on the back = contrast

PN2222 transistor and LED



Flat side facing you:
E, B, C from the left.

E to GND
B via 1 kΩ
C to buzzer −



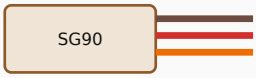
LED: long leg = +
short = −

HC-SR04 ultrasonic sensor



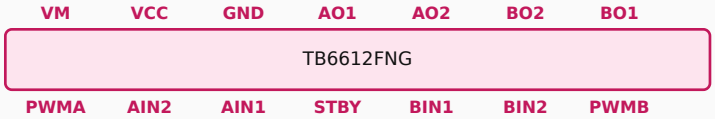
The two round “eyes” face forward.

SG90 servo: 3 wires



brown → GND
red → 5 V
orange → signal

TB6612FNG motor driver module (read the tiny labels: pin order differs between makers)



Top row = power and the 4 motor outputs.
Bottom row = signals from the UNO Q.
STBY must be wired to 3V3 or the motors stay off.

What the pins on each part are called.

Part 5: Software and the weekend

Software

The laptop is the hub: it reads the gamepad, runs the game and the single auto-pilot, and commands the robot and the tower. The three Arduino boards stay simple and fast.

Laptop program (Python 3 with pygame and pyserial)

File	Job
main.py	Starts everything and runs the game loop at 60 frames per second
gamepad.py	Reads the Bluetooth gamepad with pygame; dead zone about 8 percent; button map; prints button numbers for setup
weather_link.py	Talks to the weather station over USB serial: reads weather lines; sends virtual temperature, lives and game over
robot_link.py	Sends wheel commands to the robot over Wi-Fi; reads its scan, state and bumper hits
tower_link.py	Sends aim and fire to the tower over Wi-Fi; reads the “ball released” reply
camera.py	Reads the ESP32-CAM or phone stream and keeps the newest frame
model.py	The shared movement model, used for both the game craft and the robot’s wheel mix
autopilot.py	The single auto-pilot
hazards.py	Scenario file, debris, asteroid spawns, tower timing
lives.py	The 15 lives, the 1.5 second hit cooldown, the game-over latch and the restart
weather.py	Temperature states, the crew warnings, the 5-second log, and the optional NASA flare check
calibration.py	Stores the measured robot constants and the tower aim table
ui.py	The heads-up display, the failed-orbit animation and the screens

The serial port and the network links each read in their own thread, so a new line is acted on at once instead of waiting for the next drawn frame.

Messages between the parts

Plain text lines. USB serial runs at 115200 baud to the weather station. The robot and the tower receive the same kind of lines over Wi-Fi (UDP or TCP). Easy to read while debugging.

From to	Message	Meaning	Rate
Weather station to laptop	W,tempC,hum,state	Weather reading and alarm state (0 normal, 1 cold, 2 hot)	Every 2 seconds
Laptop to weather station	V,tempC	Virtual temperature for the simulation mode (hot on the sunlit side, cold in shadow)	As needed

Laptop to weather station	H,lives	Lives left, shown as hearts; also flashes the yellow hit LED when it drops	On change
Laptop to weather station	G	Game over: SIGNAL LOST and GAME OVER, warnings quiet	Once
Laptop to weather station	R	Reset to 15 lives	On restart
Laptop to robot	D,left,right,ap	Wheel speeds (-255 to 255) and the auto-pilot flag	50 per second
Laptop to robot	X	Game over latch: stop and ignore wheel commands	5 per second until confirmed
Laptop to robot	R	Reset: release the latch	On restart
Robot to laptop	S,angle,distCm,flags	One scan reading and stop flags	About 15 per second
Robot to laptop	B,side	Bumper hit, left or right	On contact
Laptop to tower	A,angle	Aim the tower (only if the pan servo is added)	Before each shot
Laptop to tower	F	Fire one ball	On each shot
Tower to laptop	K	Paper ball released	On each shot

Shared movement model. One speed v and one turn rate w drive both bodies: $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = \omega$. For the robot, left wheel = $v + k \cdot w$ and right wheel = $v - k \cdot w$, scaled to the motor driver's 0 to 255 range, and applied to both wheels on a side. A dead zone of about 8 percent on the stick stops drifting. Measure the robot's real speed and turn rate and put them in the game, so the two paths agree.

Auto-pilot logic (sketch)

```
def autopilot(state, hazards, stick):
    v, w = hold_orbit(state)                # keep the ring radius, fly around Earth
    v += 0.3 * stick.y                      # the player's input is a nudge
    w += 0.3 * stick.x
    near = closest_predicted_hazard(state, hazards, horizon=3.0)
    if near is None:
        return v, w
    side = clearer_side(state, hazards)     # +1 left, -1 right
    if near.distance < DANGER:              # about 25 cm
        return 0.4 * v, side * W_DODGE      # the dodge overrides the stick
    if near.distance < WARNING:             # about 40 cm
        return 0.7 * v, w + 0.5 * side * W_DODGE
    return v, w
```

Weather station firmware (untested sketch, a starting point)

Runs on the UNO R3. Pins match the wire list in Build 1. Never use delay().

```
#include <DHT.h>
#include <LiquidCrystal.h>          // the 16-pin weather LCD
#include <Wire.h>
#include <LiquidCrystal_I2C.h>      // the I2C lives display

const float COLD_C = 18.0, HOT_C = 30.0, HYST = 1.0;
enum State { NORMAL, COLD, HOT };
State state = NORMAL;

DHT dht(2, DHT11);
LiquidCrystal weatherLcd(8, 9, 10, 11, 12, 13);   // RS, E, D4, D5, D6, D7
LiquidCrystal_I2C livesLcd(0x27, 16, 2);          // try 0x3F if nothing shows
const int GREEN = 3, BLUE = 4, RED = 5, HIT_LED = A0;
const int ACTIVE_BUZZ = 6, PASSIVE_BUZZ = 7;
```

```
unsigned long tSensor = 0, tLcd = 0, tBlink = 0, tHit = 0;
float temp = 22, hum = 40;
bool simMode = false, blinkOn = false, gameOver = false;
int lives = 15;
byte fullHeart[8]  = {0,10,31,31,31,14,4,0};
byte emptyHeart[8] = {0,10,21,17,17,10,4,0};

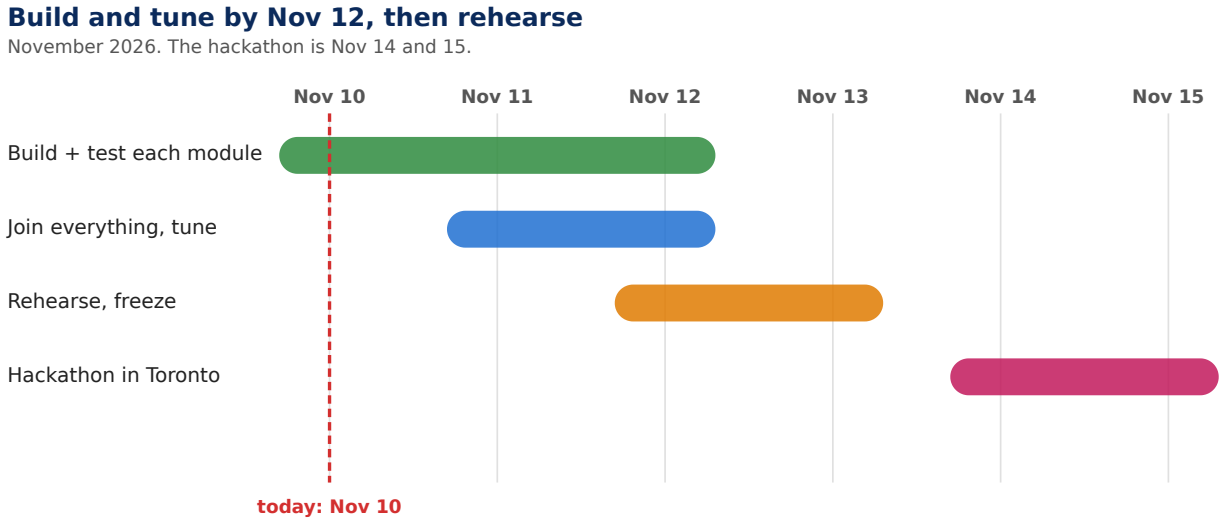
void showWeather() {                       // called every 5 s and on every state change
  weatherLcd.clear();
  if (gameOver) { weatherLcd.print("SIGNAL LOST"); return; }
  if (state == HOT)  { weatherLcd.print("!! HOT SPACE !!");  weatherLcd.setCursor(0, 1); weatherLcd.print("CREW:
HEAT WARN"); }
  else if (state == COLD) { weatherLcd.print("!! COLD SPACE !!"); weatherLcd.setCursor(0, 1);
weatherLcd.print("CREW: COLD WARN"); }
  else { weatherLcd.print("T "); weatherLcd.print(temp, 1); weatherLcd.print("C NORMAL");
         weatherLcd.setCursor(0, 1); weatherLcd.print("H "); weatherLcd.print((int)hum); weatherLcd.print("%"); }
}
void updateState() {
  State old = state;
  if (state == NORMAL) { if (temp < COLD_C) state = COLD; else if (temp > HOT_C) state = HOT; }
  else if (state == COLD && temp > COLD_C + HYST) state = NORMAL;
  else if (state == HOT  && temp < HOT_C - HYST) state = NORMAL;
  if (state != old) { showWeather(); tLcd = millis(); }     // warning shows at once
}
void outputs(unsigned long now) {
  if (now - tBlink >= 100) { tBlink = now; blinkOn = !blinkOn; }   // about 5 blinks a second
  bool quiet = gameOver;
  digitalWrite(GREEN, state == NORMAL && !gameOver);
  digitalWrite(BLUE, state == COLD && blinkOn && !quiet);
  digitalWrite(RED,  state == HOT  && blinkOn && !quiet);
  digitalWrite(ACTIVE_BUZZ, !quiet && state == HOT && blinkOn);
  if (!quiet && state == COLD && blinkOn) tone(PASSIVE_BUZZ, 400); else noTone(PASSIVE_BUZZ);
  digitalWrite(HIT_LED, now - tHit < 1000);                         // 1 second hit flash
}
void showLives() {
  livesLcd.setCursor(0, 0);
  if (gameOver) livesLcd.print("SIGNAL LOST     ");
  else { for (int i = 0; i < 15; i++) livesLcd.write(i < lives ? 0 : 1); livesLcd.print(' '); }
  livesLcd.setCursor(0, 1);
  if (gameOver) livesLcd.print("GAME OVER       ");
  else { livesLcd.print("T-Minus_6 L"); livesLcd.print(lives); livesLcd.print("/15 "); }
}
void setup() {
  Serial.begin(115200);
  dht.begin();
  weatherLcd.begin(16, 2);
  livesLcd.init(); livesLcd.backlight();
  livesLcd.createChar(0, fullHeart); livesLcd.createChar(1, emptyHeart);
  pinMode(GREEN, OUTPUT); pinMode(BLUE, OUTPUT); pinMode(RED, OUTPUT);
  pinMode(HIT_LED, OUTPUT); pinMode(ACTIVE_BUZZ, OUTPUT);
  showLives(); showWeather();
}
void handleLine(const String& s) {                // lines from the laptop
  if (s.startsWith("H,")) { int n = s.substring(2).toInt(); if (n < lives) tHit = millis(); lives = n; showLives();
}
  else if (s.startsWith("V,")) { simMode = true; temp = s.substring(2).toFloat(); updateState(); }
  else if (s == "G") { gameOver = true; lives = 0; showLives(); showWeather(); }
  else if (s == "R") { gameOver = false; lives = 15; simMode = false; showLives(); showWeather(); }
}
void loop() {
  unsigned long now = millis();
  if (now - tSensor >= 2000) {                      // read the sensor every 2 s
    tSensor = now;
    float t = dht.readTemperature(), h = dht.readHumidity();
    if (!isnan(t) && !simMode) { temp = t; hum = h; updateState(); }
    Serial.print("W,"); Serial.print(temp, 1); Serial.print(','); Serial.print((int)hum); Serial.print(',');
Serial.println((int)state);
  }
  if (now - tLcd >= 5000) { tLcd = now; showWeather(); }   // record every 5 s
  outputs(now);
  // read serial lines here and call handleLine(); no delay() anywhere
}
```

Libraries. Arduino: DHT sensor library with Adafruit Unified Sensor, LiquidCrystal (built in) for the 16-pin LCD, LiquidCrystal_I2C for the lives display, Wire and Servo (built in), AccelStepper for the robot. Python: pygame, pyserial and (for the camera) opencv-python.

The UNO Q programs (robot and tower) use the same logic, written in App Lab. How the Linux side hands messages to the microcontroller is the day-one check.

Build plan and timeline

Everything should be built and tuned by Nov 12, so Nov 13 is for the submission guides and the weekend is for polish and submitting before the Nov 15, 11:59 PM deadline.



Build timeline. Today is Nov 10.

Each module is built and tested on its own first (weather station, robot, tower, camera), the pieces come together on Nov 11 and 12, and tuning runs to Nov 12. If the rules turn out to require building during the event, keep only planning, ordering and testing of single parts before Nov 14, and move the integration into the weekend.

Suggested split (adjust to the team’s size, up to six)

- **Dhyanesh:** game, auto-pilot and the weather station (Python and Build 1).
- **Shubh:** robot motors, driver and firmware (Build 2, motors).
- **Nubaid:** robot power, wiring and the bumper (Build 2, power).
- **Ojas:** camera (Build 4) and the camera view in the game.
- **Vidhan:** asteroid tower (Build 3).
- **Aansh:** assembly, the arena board, the Earth stand and debris. Everyone helps with the submission text, demo video and pitch.

First tasks

- ☐ Decide the challenge to enter.
- ☐ Ask the Local Lead about table space, power and hardware demos.
- ☐ Gather every part at Dhyanesh’s by Nov 11 (see Who brings what).
- ☐ Set up a shared code folder and install the Arduino libraries.
- ☐ Pair the gamepad with the laptop and print its button numbers.

- ☐ Confirm each teammate has registered and joined the team page.
- ☐ Day-one UNO Q test: Wi-Fi message from the laptop to the motors.

Hackathon weekend plan

What to bring

- ☐ Laptop with the game, the Arduino IDE and App Lab already installed, and its charger
- ☐ The ELEGOO kit, both UNO Q boards, USB cable and two USB-C cables
- ☐ The robot (fully built), the 4 AA pack with 4 charged spares, the 9 V battery with a spare, both power banks charged
- ☐ The tower, paper balls (20 or more), the cardboard board, debris pieces, the Earth stand, tape
- ☐ The weather station with its breadboard, a hair dryer and a sealed ice pack
- ☐ The Bluetooth gamepad, a phone for the hotspot, an extension cord and power strip
- ☐ Spare wires, resistors, LEDs, zip ties, hot glue, a small screwdriver, scissors
- ☐ This document (printed) and the wire lists

Saturday Nov 14

Time	What
9:00	Arrive, find the table, check power outlets. Open the Project tab on the team page.
9:30 to 10:30	Set up the board and the arena. Tape the orbit ring and start mark. Place the Earth stand.
10:30 to 12:00	Plug in and test each machine one at a time: weather station, then robot (wheels on the table), then tower, then camera. Use the Test it lists.
12:00 to 1:00	Lunch. Recharge power banks.
1:00 to 4:00	Join everything. Pair the gamepad, start the hotspot, run the game with the robot. Calibrate speed, turn rate and tower aim.
4:00 to 6:00	Auto-pilot tuning, weather warnings, lives and game over. Fix problems.
Evening	Run the full demo 3 times in a row. Write the project text. Charge everything.

Sunday Nov 15

Time	What
Morning	Quick hardware check, then record the demo video (screen, robot and hands only; no faces).
Midday	Write and review the submission text. Rehearse the 3-minute demo.
Afternoon	Judging, if the local event runs it. Be ready to demo.
Before 11:59 PM	**Submit** from the Project tab. Do not wait until the last minute.

If something breaks

Problem	First thing to try
Nothing lights up on the weather station	Check USB cable and the 5V and GND wires on the breadboard rails
LCD shows boxes or nothing	Turn the contrast knob. Check RS, E and the four data pins

I2C display blank	Turn the blue screw; run the I2C scanner; try address 0x3F
Robot wheels do nothing	Is STBY connected to 3V3? Is the AA pack switch on? Is VM connected?
One side drives backward	Swap the red and black motor wires on that side
Robot resets when the motors start	Check the 100 µF capacitor and that the UNO Q is on the power bank, not the AA pack
Robot does not answer over Wi-Fi	Same hotspot? Fall back to the USB-C cable
Distance reading is always 0	Check the ECHO divider and the sensor’s 5V from the MB102
Camera picture lags	Smaller picture, fewer frames, check that the hotspot is not crowded

Risks, testing and the demo

The biggest risks are timing (days left), the robot drifting away from the game, and the sensors being asked to do more than they can. Each has a fix below.

Risks and fixes

Risk	Why it matters	Fix
Robot and game drift apart	The robot tracks position by adding up commands, so errors grow	A sync button that puts the robot on its start mark and resets the game; slow speeds; calibrate the constants; optional webcam
Ultrasonic sensor misses things	Narrow beam, slow stepper sweep, cannot see a bare sphere or a small fast paper ball	Flat-faced debris at least 8 cm tall; a cylinder for Earth; keep the robot slow; the laptop predicts the paper balls from the fire command
Ultrasonic does not work in space	Sound needs air	Say it before a judge asks: it stands in for the radar or lidar a real spacecraft would use
Wi-Fi lag or dropouts	The robot would feel late	USB-C cable fallback; a 300 ms stop if commands stop; own hotspot; measure the delay early
Motor power resets the UNO Q	Motors draw spikes	Separate supplies, joined ground, a 100 µF capacitor across VM, a 0.1 µF on each motor
Paper balls miss or go too fast	Launch speed and aim vary	A shallow ramp, an aim table from test shots, and a generous hit area
Out of time	Several working parts	Build in this order: game and gamepad, robot driving, weather station, auto-pilot, tower
Wrong challenge or rule surprise	Rules for earlier work and hardware are not in the public FAQ	Read the Project Submission guide on Nov 13 and ask the Local Lead
UNO Q is new to the team	App Lab, how Wi-Fi messages reach the motor microcontroller, whether AccelStepper and Servo work on it, exact PWM pins and 3.3 V logic	Test Wi-Fi, a motor and the stepper on the first day. Fallback: a USB-C cable to the laptop
Camera delay and Wi-Fi traffic	Video can lag and slow control messages	Small picture, newest frame only, own hotspot, test steering and video together
Wrong pin order on a part	DHT11 and PN2222 pins differ between makers	Read the printed labels; use the pin cheat sheet

Test checklist

- ☐ Left on the stick turns left on screen and on the robot, 20 times in a row.
- ☐ Stick to wheel delay measured and written down; the robot stops 300 ms after the link is cut.
- ☐ Robot stops within 8 cm of an obstacle even with the laptop off.
- ☐ Auto-pilot dodges a debris piece in 10 of 10 runs in the game and 8 of 10 on the table.
- ☐ 10 paper balls fired: how many land in the lane, and how long they take.
- ☐ Weather: hot, cold and normal each give the right LED, sound and display text; the warning shows at once; the display logs every 5 seconds; the two buzzers sound different.
- ☐ The I2C display shows up in an I2C scan. Each hit takes exactly one heart, and a second hit within 1.5 seconds takes none. At zero lives the robot stops within 300 ms and ignores the stick; Start restores 15 hearts.
- ☐ Run the full demo 3 times in a row without a restart.
- ☐ Camera (A or B): the picture appears in the game; measure the delay with a phone stopwatch. Steering still feels instant with the video running.

Demo plan for the judges (about 3 minutes)

1. The problem in one line: debris and temperature swings threaten spacecraft.
2. Manual run: steer by gamepad; screen and robot move together.
3. An asteroid is launched; the player dodges by hand, then misses one on purpose to show a hit and a heart disappearing.
4. Switch on auto-pilot: the robot and T-Minus_6 dodge debris and the next asteroid by themselves.
5. Weather: blow hot air at the sensor; the crew warning appears, the red LED and buzzer blink fast; cool it; the blue LED and the other buzzer; back to green.
6. Close with what was built, what was measured, and what comes next.

The demo video follows the rule about minors: show the screen, the robot and hands only, and use captions or an adult voice-over.

Open decisions and sources

These decisions are still open. Each has a suggestion so the team can settle them in one meeting.

Decision	Options	Suggestion
Challenge to enter	Keep “Build a Junior Astronaut Mission Trainer” (frame it as a trainer that teaches players to pilot, dodge debris and handle heat and cold), or switch to “Space Mission Design Game”	Read both full challenge statements (released Oct 28) and pick the one whose deliverables match. They have not been checked for this document
Who wins, stick or auto-pilot	The dodge overrides the stick in the danger zone (proposed), or the stick always wins	Dodge overrides, so auto-pilot visibly protects the craft
Temperature thresholds	Cold below 18 C and hot above 30 C (starting values), or the room’s starting temperature plus or minus 5 C	Relative thresholds are safer, because the venue’s room temperature is unknown. Decide at the first sensor test
Robot link	UNO Q Wi-Fi through a hotspot, or a USB-C cable	Wi-Fi, with the USB-C cable as the fallback
Camera	ESP32-CAM or a normal phone	Whichever arrives first and gives the smaller delay in a test

Robot position	Dead reckoning with a sync button, or an overhead webcam with a marker	Dead reckoning first; webcam only if time remains
Orbit size	40 cm ring (needs about 110 cm of table) or 30 cm ring	Decide once the venue’s table size is known
Work started before the event	Allowed or not under the Space Apps rules	Read the Project Submission guide on Nov 13 and ask the Local Lead now
Tower aiming	One servo and a fixed lane, or add a second servo to pan	One servo first; add the pan servo only if time remains
I2C display address	0x27 or 0x3F	Run the I2C scanner on day one
NASA data hook	Use real solar flares from the DONKI service to trigger heat events, or leave it out	Add it only after the core demo works

Sources

- NASA Space Apps Challenge participant FAQs: 2026 key dates, age and parent rules, rules for videos and submissions.
- The team’s own Space Apps page, as seen by the team: team size limit, the Project submission window and certificate, and the Nov 30 survey.
- The ELEGOO UNO R3 kit’s printed parts sheet: the kit contents.
- NASA Open APIs at api.nasa.gov, including the DONKI space-weather service: not opened for this document, so check its documentation before using it.

Part specifications, prices, pin assumptions for the UNO Q, and timing figures in this document are approximate, from general knowledge. Check them against datasheets, current prices and real measurements before relying on them.